How ALF was ported to GPUs

The implementation of the auxiliary-field QMC sweep on NVIDIA, Intel and AMD accelerators

Branch `gpu-port-unified`, state of 2026-11-13

[author]

2026-11-13

Contents

1 Reader's guide and summary **5**

1.1 What this document is . . . 5
1.2 How to read it . . . 5
1.3 Summary of the implementation . . . 5
1.4 How the port came about . . . 6
1.5 Notation . . . 7

2 The sweep as ALF does it on the CPU **7**

2.1 Propagators and the Green function . . . 7
2.2 Wrapping . . . 8
2.3 Stabilisation with UDV stacks . . . 8
2.4 Local updates . . . 8
2.5 Measurements . . . 9
2.6 Measured CPU cost per part . . . 9
2.7 The production model . . . 10
2.8 What this means for a GPU . . . 10

3 Design decisions and why **11**

3.1 Constraints . . . 11
3.2 Programming model: OpenMP target offload plus vendor libraries . . . 11
3.3 One source tree, backend by preprocessor . . . 12
3.4 Device-resident Green function and stacks . . . 12
3.5 Plain arrays instead of ALF's derived types . . . 13
3.6 What "host-assisted" means . . . 13
3.7 The library layer `alf_la` . . . 13
3.8 What device code looks like . . . 15

4 Data model **16**
4.1 The sweep on the device: who does what . . . 16
4.2 How residency is expressed . . . 17
4.3 The constant mirror of the model . . . 17
4.4 Work arrays . . . 18
4.5 Memory footprint . . . 19
4.6 All host–device transfers in one sweep . . . 19

5 Propagation on the device **20**
5.1 Hopping: five routines, three code paths . . . 21
5.2 Time-dependent hopping (the point defect) . . . 22
5.3 Interaction vertices . . . 22
5.4 Composite operations . . . 23
5.5 Operation counts . . . 23

6 Stabilisation on the device **23**
6.1 Stacks and slots . . . 23
6.2 The QR decomposition with pre-pivoting . . . 24
6.3 Decompose (default stabilisation) . . . 24
6.4 Decompose with `STABLOG` . . . 25
6.5 Assembling G on the device (`CGR`) . . . 26
6.6 Precision monitoring . . . 26
6.7 The phase of the determinant . . . 26
6.8 Vendor QR and own device QR . . . 27
6.9 Transfers of the stabilisation . . . 29

7 Local updates **29**
7.1 Why the rank-one update is slow on a GPU . . . 29
7.2 Common part: the ratio and the decision . . . 30
7.3 Standard (rank-one) update . . . 30
7.4 Delayed update . . . 30
7.5 Submatrix update . . . 31
7.6 Block mode . . . 31
7.7 Synchronisation points per block . . . 33
7.8 Launch-latency budget of a block . . . 33
7.9 The asynchronous block path on Intel . . . 33
7.10 What remains: the flush . . . 34

8 Observables **34**
8.1 Hook-up and conditions . . . 34

8.2 What is measured . . . 35
8.3 How the reductions are organised . . . 35
8.4 The time-displaced pass . . . 36

9 MPI, parallel parameters and sharing a GPU 37
9.1 What MPI does in a GPU run . . . 37
9.2 Parallel parameters and tempering . . . 38
9.3 Several ranks on one GPU . . . 38
9.4 The production layout . . . 39

10 The three backends in detail 39
10.1 NVIDIA . . . 39
10.2 Intel . . . 40
10.3 AMD . . . 41
10.4 What would a fourth backend need . . . 42

11 Verification 42
11.1 The method that worked . . . 42
11.2 What did not work, and why . . . 43
11.3 Current results per backend . . . 44

12 Performance 44
12.1 Elementary operations . . . 45
12.2 Time per sweep against L . . . 45
12.3 Update scheme and N_{d} . . . 46
12.4 `Nwrap` . . . 47
12.5 Where the time goes . . . 47
12.6 Counting the operations of one sweep . . . 48
12.7 Throughput per device . . . 48
12.8 Tuning rules . . . 48

13 What is not ported, and what it would take 49
13.1 The refusal list . . . 49
13.2 What each item would take . . . 50
13.3 Ranked speedup candidates . . . 50

14 Lessons for a GPU port of the whole of ALF 51

A Glossary of GPU terms for a QMC physicist 52

B File-by-file guide to the GPU source 53

C Build lines **55**

D Run-time switches **56**

1 Reader's guide and summary

1.1 What this document is

This document describes how the finite-temperature sweep of ALF was moved onto graphics processors (GPUs). It is written for a reader who knows the auxiliary-field QMC algorithm and the CPU code of ALF well, but who has not programmed a GPU. It complements the shorter report `docs/gpu/report/gpu_port_report.tex`, which summarises status, speed and open items. That report is not repeated here. This document explains the implementation: which data live where, which mathematics each device routine evaluates, why the code is organised the way it is, and what each design choice cost or gained.

The ground truth is the source code of the branch `gpu-port-unified` as of 2026-11-13. File and line references are given as `file:line`. The Markdown notes in `docs/gpu/` are used for history, measurements and decisions. Every number taken from a note carries a LaTeX comment with the source file next to it. Where the code and a note disagree, the text follows the code and a framed box records the disagreement. Items that could not be determined are marked **[TODO: like this]**.

1.2 How to read it

- Section 2 restates the CPU sweep in the notation of the ALF documentation and gives the measured CPU cost of each part. It says which operations had to move and why.
- Section 3 gives the design decisions: the programming model, the library layer `alf_la` with a table of every routine and its implementation per vendor, and the principle of a device-resident Green function.
- Section 4 lists every device array, its size as a function of N, L_{trot}, N_{FL} and `Nwrap`, and every transfer between host and device in one sweep.
- Sections 5 to 8 are the core: propagation, stabilisation, local updates and measurements on the device, each with the formulas as coded.
- Section 9 covers MPI, parameter-parallel runs and sharing a GPU between ranks.
- Section 10 describes the three vendor backends, the problems met and their solutions.
- Sections 11 and 12 give verification and performance.
- Sections 13 and 14 say what is not ported, what it would take, and what we learned for a port of the whole of ALF.
- The appendix has a glossary of GPU terms (Appendix A); readers new to GPUs may want to read it first. It also has a file-by-file guide to the GPU source and the build lines.

1.3 Summary of the implementation

What runs where. In a GPU build the following parts of the sweep run on the accelerator (the *device*); the CPU is called the *host*.

- The Green function G of every flavour lives on the device for the whole sweep. The host copy is updated only when host code needs it (ticket 208-1, `gpu_update_mod.F90:52-53`).
- Propagation $G \to BGB^{-1}$ by the hopping and the interaction vertices: own kernels for checkerboard groups and vertices, vendor matrix products for dense hopping.
- Stabilisation: the UDV stacks, the QR decompositions, the assembly of G from the left and right decompositions (`CGR`), the precision check and the phase of the determinant.
- Local updates: the gathers of matrix elements and the flushes of the delayed and submatrix schemes. The proposal, the ratio arithmetic on small matrices, the Metropolis decision and the random numbers stay on the host, so that the GPU run follows the same Markov chain as the CPU run with the same seed.
- Equal-time observables of the Hubbard model (including the point-defect observables) and the

time-displaced pass with its observables.

How. Own kernels are written as OpenMP `target` regions in the existing Fortran files. All dense linear algebra goes through one new module, `alf_la`, whose routines call cuBLAS/-cuSOLVER (NVIDIA), oneMKL (Intel) or hipBLAS/hipSOLVER and own HIP kernels (AMD), or plain BLAS/LAPACK in a CPU build. The backend is selected by `configure.sh GPU=none|nvidia|intel|amd`. A build with `GPU=none` is bitwise identical to upstream ALF (with `LOBS_ST=1`; `CONSOLIDATION.md`). The GPU code is in seven new modules `Prog/gpu_*.F90` (about 12 500 lines) plus the library layer (about 5 900 lines of Fortran, C++, SYCL and HIP). The CPU code is touched only by guarded early branches.

The four ideas that made it fast.

1. **Keep G and the stacks on the device.** Moving an $N \times N$ complex matrix across PCIe costs more than most operations on it. Only scalars, short vectors and small blocks cross the bus during a sweep (Section 4).
2. **Replace the rank-one update by the submatrix update in block mode.** One sequential rank-one update per site is bound by the latency of the device (about 10 to 30 μs per operation), not by its arithmetic. The block mode moves all per-site work to a small host matrix and leaves the device two matrix products per block (Section 7). On the A30 this is 2.4 to 5 times faster than the standard update.
3. **Replace pivoted QR by pre-pivoted QR.** No GPU library has a fast QR with column pivoting. Sorting the columns by their norms once and then doing an unpivoted QR gives the same precision for the default stabilisation (Section 6).
4. **Write own kernels where the vendor library is weak.** oneMKL's QR runs its panels on the host, rocSOLVER's QR launches thousands of tiny kernels, and the OpenMP runtime of GCC for AMD costs 150 μs per launch. Own QR kernels in SYCL and HIP, HIP kernels for the small operations, an asynchronous SYCL path for the update blocks, and a patched GCC plugin removed these bottlenecks (Section 10).

Results. For the production model (Hubbard model with a space-time point defect, $\beta = L$, $\Delta\tau = 0.1$, $N = L^2$) one sweep at $L = 24$ takes 3.5 to 5.3 s on one GPU (H100 3.51, Max 1550 tile 4.04 to 4.96, MI210 4.41, A30 5.25). On one CPU thread the same build takes 148.6 s (A30 host, `TUNING_DEFECT.md`). One chain does not fill a GPU; with several chains per GPU (MPS on NVIDIA, CCS on Intel) the throughput per GPU rises by a factor 2 to 4 (Section 9). The GPU runs reproduce the CPU runs of the same compiler and seed to 10^{-13} to 10^{-15} in the observables for `Mz=.F.` (Section 11).

Limits. The port covers one model family: Hubbard-type models with diagonal single-site vertices and discrete fields, finite temperature, sequential local updates. Everything else is refused at setup and runs on the CPU (Section 13).

1.4 How the port came about

The chronology matters for reading the code, because several comments still describe intermediate states.

- **Phase 0 and 1** (to 2026-10-27): CPU profile and baseline, kernel micro-benchmarks on the A30, prototypes of OpenMP and OpenACC launch costs; ADR-001 (programming model) and ADR-002 (`alf_la` and data model) accepted at the Phase 1 gate on 2026-10-27 (`PLAN.md`, section 5).
- **Phase 2, tickets 201 to 208** (2026-10-27 to 2026-11-03): the NVIDIA minimum viable product. 201–206: `alf_la`, the mirror, wraps, rank-one update, stabilisation; 207: the τ pass; 208-1: G device-authoritative; 208-2 and 208-3: equal-time and τ observables on the device.

Ticket 212 made all reductions deterministic. This phase was slowed by the verification process described in Section 11, until the owner's decision 41 of 2026-11-02.

- **After 2026-11-02** (about seven days, `EVAL_ORIGINAL_GPU_BRANCH.md`): CPU delayed and submatrix updates taken into the branch and ported to the device with block mode; time-dependent hopping for the point defect; MPI and parameter-parallel runs; H100 on Helma; Max 1550 production on SuperMUC-NG Phase 2 with the staging buffer, the asynchronous block path and the own SYCL QR; the AMD backend with HIP kernels and own HIP QR.
- **2026-11-13**: the vendor lines were merged with `-no-ff` into `gpu-port-unified` (`CONSOLIDATION.md`): first the parameter-parallel and optimisation lines of the NVIDIA work, then the Intel production and team lines, then AMD. Conflicts were in `alf_la_mod.F90`, `gpu_lowrank_mod.F90`, `gpu_obs_mod.F90`, `configure.sh` and job files. In `gpu_lowrank_mod.F90` the Intel kernels are the base and the HIP calls are applied under `ALF_LA_HIP`; in `gpu_obs_mod.F90` the summation order of NVIDIA and Intel was kept and AMD uses its HIP kernels. The LU-free phase was added the same day.

1.5 Notation

N	number of orbitals, `Ndim`; $N = L^2$ for the square lattice of side L
L_{trot}	number of time slices, $\beta/\Delta\tau$
N_{FL}	number of flavours; $N_{\mathrm{FL,eff}}$ the number actually computed (`Calc_Fl`)
N_{d}	`N_delay`, the maximal rank of a pending low-rank correction
N_{ST}	number of stabilisation points, `NSTM`, about L_{trot}/`Nwrap`
B_τ	propagator of slice τ, $B_\tau = e^{-\Delta\tau K}\prod_n e^{V_n(\sigma_{n,\tau})}$ (order as in ALF)
$B(\tau_2,\tau_1)$	$B_{\tau_2}\cdots B_{\tau_1+1}$
$G(\tau)$	equal-time Green function $\langle c_i c_j^\dagger\rangle$ at slice τ, $[1+B(\tau,0)B(\beta,\tau)]^{-1}$
D→H, H→D	copy from device to host, from host to device

2 The sweep as ALF does it on the CPU

This section restates the finite-temperature sweep of `Prog/main.F90` in the notation of the ALF documentation. Its purpose is to name every operation that the GPU code replaces, with its cost, so that the later sections can refer to it.

2.1 Propagators and the Green function

For a given configuration of the auxiliary fields $\sigma_{n,\tau}$ the fermion weight is $\prod_f \det[1+B_f(\beta,0)]^{N_{\mathrm{SUN}}}$, with one propagator per flavour f,

$$B(\tau_2,\tau_1)=\prod_{\tau=\tau_1+1}^{\tau_2} B_\tau, \qquad B_\tau = e^{-\Delta\tau T}\prod_n e^{\,g_n\,\eta(\sigma_{n,\tau})\,V_n}\,, \tag{1}$$

where the order of the factors is that of `Op_T` and `Op_V` in the Hamiltonian. The hopping exponential $e^{-\Delta\tau T}$ is either one dense matrix or a product of *checkerboard* factors, one per element of `Op_T`, each acting on a few sites (`Hop_mod.F90`). With `Symm=.T.` the hopping is split symmetrically, $e^{-\Delta\tau T/2}\,V\,e^{-\Delta\tau T/2}$, and observables are measured on the transformed Green function $e^{\Delta\tau T/2}\,G\,e^{-\Delta\tau T/2}$ (`Hop_mod_Symm`).

For the Hubbard model each vertex $\mathrm{Op}_V(n)$ acts on one site p_n with $N_{\text{non-zero}}=1$, so e^{V_n} is the identity except for one diagonal element $e^{\alpha\,\sigma}$. The equal-time Green function at slice τ is

$$G(\tau)=\left[1+B(\tau,0)\,B(\beta,\tau)\right]^{-1}. \tag{2}$$

2.2 Wrapping

Between stabilisations, G is moved from slice τ to $\tau+1$ by a similarity transformation,

$$G(\tau+1) = B_{\tau+1}\, G(\tau)\, B_{\tau+1}^{-1}, \tag{3}$$

and in the downward direction by the inverse. In ALF this is `WRAPGRUP` and `WRAPGRDO` (`Wrapgr_mod.F90`). They are interleaved with the local updates: on the way up, the hopping is applied (`Hop_mod_mmthr`, `Hop_mod_mmthl_m1`), then for every vertex n in order the vertex factor of the old field is wrapped in (`Op_Wrapup`) and the field on that vertex is updated. On the way down the order is reversed. The wrap of one vertex is a scaling of row p by z_a and column p by $z_b = 1/z_a$:

$$G_{rj} \to z_a G_{rj}, \quad G_{jr} \to z_b G_{jr} \qquad (r = p_n). \tag{4}$$

The hopping part costs two N^3 products per slice with dense hopping, or $O(N^2)$ with checkerboard factors.

2.3 Stabilisation with UDV stacks

Products of many B_τ are ill conditioned: their singular values span many orders of magnitude. ALF therefore keeps the products in factorised form,

$$B(\tau,0) = U_R D_R V_R, \qquad B(\beta,\tau) = V_L^\dagger D_L U_L^\dagger, \tag{5}$$

with U unitary, D diagonal with the scales, and V well conditioned (upper triangular times a permutation). Every `Nwrap` slices, `WRAPUR` (or `WRAPUL`) multiplies the next `Nwrap` propagators onto UD and re-factorises with a QR decomposition with column pivoting (`UDV_mod`, `QDRP_decompose`, LAPACK `ZGEQP3`):

$$B(\tau_2,\tau_1)\, UD = QRP^T, \qquad U \leftarrow Q,\ D \leftarrow |\mathrm{diag}\, R|,\ V \leftarrow D^{-1}RP^TV. \tag{6}$$

The decompositions at the stabilisation points are kept in a stack `udvst`(N_{ST} slots per flavour). On the upward pass the right decompositions are built and the left ones read from the stack; on the downward pass the reverse. With `STABLOG` the scales are kept as logarithms.

At each stabilisation point G is computed afresh from the two decompositions (`CGR`, `cgr1_mod.F90`):

$$G = \left[1 + U_R D_R V_R V_L D_L U_L^\dagger\right]^{-1} = U_L \left[U_R^\dagger U_L + D_R V_R V_L D_L\right]^{-1} U_R^\dagger. \tag{7}$$

The bracket is formed with the scales applied on the outside and is again inverted by a pivoted QR. The phase of $\det G$ is tracked from the same factors. `Control_PrecisionG` compares the fresh G with the wrapped one and stops the run when the difference exceeds a threshold.

2.4 Local updates

For a proposed change of the field on vertex n at slice τ, with $\Delta = e^{\,g\,(\eta_{\mathrm{new}}-\eta_{\mathrm{old}})\,V} - 1$ on site p, the ratio of weights is

$$R = \prod_f \left[1 + \Delta_f\,(1 - G_{pp})\right]^{N_{\mathrm{SUN}}}\ \frac{\gamma_{\mathrm{new}}}{\gamma_{\mathrm{old}}}\, e^{\cdots}, \tag{8}$$

and on acceptance (`Upgrade2`, Sherman–Morrison)

$$G' = G - \frac{\Delta}{1 + \Delta(1 - G_{pp})}\, G_{:,p}\,(e_p^T - G_{p,:}). \tag{9}$$

This is one `ZGERU` of size $N \times N$, $8N^2$ flops and $32N^2$ bytes of memory traffic per accepted flip, and there are $2\,N\,L_{\mathrm{trot}}$ proposals per sweep (each vertex on the way up and on the way

down). The branch also contains CPU versions of the *delayed* and *submatrix* schemes (`Lowrank_update_mod.F90`, parameter `Sequential_scheme`; ported on 2026-11-06 from the fork branch `feature/delayed-submatrix-clean`, commit `80c1c80d`, and listed in `GPU_PLAN_V2.md` as a prerequisite of an upstream merge), which collect up to N_d accepted flips and apply them as one matrix product (Section 7).

2.5 Measurements

Equal-time observables are measured on every slice from $G(\tau)$ by the Hamiltonian's `Obser` routine. For the Hubbard model these are the kinetic, potential and total energy, the density, and the equal-time correlation functions (Green, spin, density) accumulated as functions of the distance $i - j$ on the lattice. With `Ltau=1`, a second pass after the sweep computes $G(\tau, 0)$, $G(0, \tau)$ and $G(\tau, \tau)$ for all τ and calls `ObserT`. That pass has its own stabilisation, `CGR2_2`, which inverts a $2N \times 2N$ block matrix with a pivoted QR.

2.6 Measured CPU cost per part

Table 1 shows where one CPU core spends its time. The numbers are from stack sampling of upstream ALF (GNU 13.3, OpenBLAS, one rank, one thread) for the plain Hubbard model ($U = 4$, $\beta = 4$, $\Delta\tau = 0.1$, `Nwrap`=10, `Ltau=1`) on three lattices with checkerboard (cb) and dense hopping (`PROFILE.md`). They are for the reference model of the profile, not for the point-defect model.

Table 1: Share of wall time in percent per routine on one CPU core (`PROFILE.md`, section "Result"). $N = 144, 400, 1024$.

	checkerboard			dense		
Routine	12^2	20^2	32^2	12^2	20^2	32^2
`CGR2_2` (τ pass, $2N \times 2N$)	27.7	29.4	23.6	19.1	16.5	14.4
`CGR`	12.7	15.0	11.2	7.4	8.8	7.1
UDV decompose	6.6	11.0	7.5	5.5	6.7	4.9
`Upgrade2` rank-1 (`ZGERU`)	17.2	23.5	33.3	9.7	15.2	20.1
`Upgrade2` ratio	3.2	1.0	0.4	1.7	0.6	0.3
`Op_Wrapup/do`	2.1	0.8	1.2	1.4	0.4	0.6
`WRAPUR/UL` B products	4.5	2.0	5.8	14.7	12.0	13.4
`PROPR/PROPRM1`	4.6	2.0	4.7	12.4	13.6	14.3
Hopping in `WRAPGR`	3.0	1.6	2.8	14.5	16.8	14.3
`ObserT`	5.0	3.9	1.9	4.3	2.2	1.2
`Obser`	11.9	7.6	6.2	7.4	4.1	3.9
Stabilisation (sum)	47.0	55.4	42.3	32.0	32.0	26.4
Local updates (sum)	22.5	25.3	34.9	12.8	16.2	21.0
B products (sum)	12.1	5.6	13.3	41.6	42.4	42.0
Measurements (sum)	16.9	11.5	8.1	11.7	6.3	5.1
Whole τ pass	41.3	40.2	34.3	40.8	36.5	35.3
Wall time per sweep (s)	0.60	8.0	193	0.80	13.7	294

The call counts per sweep at 12^2 with checkerboard are: `upgrade2` 11 520, `op_wrap` 46 080, `cgr` 16, `cgr2_2` 8, `decompose` 24, `obser` 79, `obsert` 41 (`PROFILE.md`). By kind of leaf routine, 23 % of the time is BLAS-3, 35 % BLAS-1/2 and 31 % plain Fortran loops of ALF (`PROFILE.md`, 12^2 cb).

2.7 The production model

The port was driven by one model: the Hubbard model on the square lattice with π flux and a space-time point defect (`point_defect_math.tex`; the predefined `Hamiltonian_Hubbard` with `ham_h0`$\neq 0$):

$$H = -t \sum_{\langle ij \rangle \sigma} (e^{iA_{ij}} c^\dagger_{i\sigma} c_{j\sigma} + \text{h.c.}) + U \sum_i (n_{i\uparrow} - \tfrac{1}{2})(n_{i\downarrow} - \tfrac{1}{2}), \tag{10}$$

with $N = L^2$ sites and discrete Ising auxiliary fields; the production parameter file has $N_{\mathrm{FL}} = 2$ and $N_{\mathrm{SUN}} = 2$ (`TUNING_DEFECT.md`), and the pinning field requires $N_{\mathrm{FL}} = 2$. The defect is a pinning field on one site at one time slice. In the code (`Hamiltonian_Hubbard_smod.F90:495-525`) it is a second hopping element per flavour, `Op_T(2,nf)`, diagonal with $\mp h_0$ on the pin site $N/2 - L/2$, with a time-dependent coupling $g_t(\tau)$ that is -1 on the first slice and 0 elsewhere. So the propagator of slice 1 carries the extra factor $e^{\pm h_0 n_{p\sigma}}$, a pulse of strength h_0 in S^z_p at $\tau = 0$. The static hopping `Op_T(1,nf)` is a dense $N \times N$ matrix; the `Checkerboard` flag is ignored when the pin is on. The production parameters are `Mz=.F.`, $U = 5$ (and a scan 5.15 to 5.425), $h_0 = 1$, `Symm=.T.`, $\Delta\tau = 0.1$, $\beta = L$ (so $L_{\mathrm{trot}} = 10L$), `Ltau=0`, `Nwrap`=10 (`TUNING_DEFECT.md`; `DECISIONS.md`, item 47). The new observables are `SpinZ_local` and `SpinZ_localtau`, the response $\langle S^z_j(\tau) \rangle$ to the pulse.

For the GPU the model has three consequences: the hopping is dense, so the wraps are matrix products (good for a GPU); the defect is a time-dependent hopping element, which needed its own device path (Section 5); and the vertices are single-site and diagonal, which makes the submatrix update in block mode possible (Section 7).

Code and documents disagree. `point_defect_math.tex` places the pulse on the two slices $L_{\mathrm{trot}}/2$ and $L_{\mathrm{trot}}/2 + 1$ with $g_t = -\frac{1}{2}$ each, and recommends the projector code and `Mz=.T.`. The code puts the pulse on slice 1 with $g_t = -1$ (after the merge of 2026-11-09), `Ham_Set` refuses the pin with the projector, and production uses finite temperature with `Mz=.F.`.

2.8 What this means for a GPU

A GPU is fast for large dense matrix products and for simple loops over many independent elements, and slow for anything sequential or small (Appendix A). Read with that in mind, the profile says:

- **Stabilisation** is the largest group (26 to 55 %). It consists of QR factorisations, triangular solves and matrix products of size N or $2N$. These map well to vendor libraries, except that *pivoted* QR (`ZGEQP3`) does not exist on GPUs (Section 6).
- **Local updates** grow with N (17 to 33 % for the rank-one update alone). A rank-one update is memory bound and strictly sequential: each proposal needs G_{pp} after the previous acceptance. On a GPU each step costs at least one kernel launch and one transfer of G_{pp} to the host, 10 to 30 μs, while a CPU core does the whole update at $N = 144$ in about 7 μs (`BENCHMARKS.md`, Phase 1: rank-1 update 4.7 μs device and 6.8 μs host at $N = 144$). This is the hardest part of the port (Section 7).
- B **products** (wraps) are 6 to 42 %: large with dense hopping, small with checkerboard. Dense hopping is a matrix product and maps to the library. Checkerboard needs own kernels (Section 5).
- **Measurements** are 5 to 17 %. They read G on every slice. If they stayed on the host, G would have to cross the bus $2L_{\mathrm{trot}}$ times per sweep, which at $N = 576$ is about 5 MB per copy and flavour. They had to move too (Section 8).

So no single part dominates: a port that moves only the matrix products to the GPU gains little, and every part had to move. That is why the port keeps all large data on the device (Section 4).

For the production model the hopping is a dense matrix (the `Checkerboard` flag is ignored

when the pinning field is on, `TUNING_DEFECT.md`), so the CPU sweep is dominated by N^3 matrix products. One CPU thread needs 1.68, 12.7 and 148.6 s per sweep at $L = 12$, 16 and 24 with $\beta = L$ (nvfortran, submatrix $N_\mathrm{d} = 8$, `TUNING_DEFECT.md`). A fit gives $t \propto L^{6.44}$ (`report/sections/s6_performance.tex`): N^3 per operation times $L_\mathrm{trot} \propto L$.

3 Design decisions and why

3.1 Constraints

The brief of the port fixed four constraints (`ADR-001-programming-model.md`, "Context"; `CLAUDE.md`):

1. One Fortran source tree for NVIDIA and Intel GPUs (AMD was added later). The backend is chosen when the code is configured.
2. A build with `GPU=none` must behave exactly like upstream ALF and must not become slower.
3. No polymorphism or deep copies of derived types on the device; data stay on the device across the sweep; every host–device transfer is documented.
4. The default and the `STABLOG` stabilisation are the targets.

The compilers set the second boundary. nvfortran (NVIDIA HPC SDK) supports OpenMP offload, OpenACC, CUDA Fortran and offload of `do concurrent`. ifx (Intel) supports OpenMP offload but not OpenACC. gfortran supports OpenMP offload to AMD GPUs (`ADR-001`).

3.2 Programming model: OpenMP target offload plus vendor libraries

The decision (ADR-001, accepted 2026-10-27 at the Phase 1 gate) has two parts.

1. **Own kernels are OpenMP 5.x `target` regions**, written in the Fortran modules next to the data they touch. A conservative subset is used that both nvfortran and ifx compile correctly: `target enter/exit data`, `target update`, `target teams distribute parallel do` over explicit-shape arrays, `use_device_addr`, and `interop` only inside the library layer. Not used on the device: `declare mapper`, unified shared memory, the `loop` construct, polymorphism, allocation, type-bound procedures (`ADR-001`, "Decision", item 1).
2. **The cubic work goes to vendor libraries** behind one module, `alf_la` (ADR-002): cuBLAS and cuSOLVER on NVIDIA, oneMKL on Intel, hipBLAS and hipSOLVER on AMD, plain BLAS and LAPACK in the CPU build.

Why not the alternatives. The ADR lists them with the reason for rejection (`ADR-001`, "Alternatives considered"):

- *OpenACC*: NVIDIA only in practice (ifx has none), so a second set of directives would be needed for Intel. A measurement on the A30 found one real advantage, the asynchronous launch: 2.7 μs for an empty asynchronous OpenACC kernel against 11.0 μs for OpenMP `nowait`. For synchronous launches, library calls and a host decision there was no gain. Even with a device-side decision the single-site update would have stayed about two times slower than a 16-core node at $N = 144$. The lever had to be algorithmic (delayed and submatrix updates), which works in either model (`ADR-001`, item 6).
- *CUDA Fortran*: NVIDIA only and more invasive than OpenACC.
- *`do concurrent` offload* (`-stdpar`): relies on unified memory, gives no control over residency and transfers, and the interplay with libraries was unclear.
- *A rewrite of the hot path in SYCL, C++ or Kokkos*: a large rewrite of Fortran code built on class-based Hamiltonians, against the spirit of one source tree. C++ is limited to small shims (SYCL for oneMKL, HIP for AMD).
- *MAGMA* for pivoted QR: NVIDIA and AMD only, and its `zgeqp3` is bound by matrix–vector

work. Kept as an option, not used.

What the decision cost later. OpenMP alone was not enough on two of the three vendors (`GPU_PLAN_V2.md`, section 2). ifx executes `nowait` target tasks undeferred (they cost 97 to 103 % of a synchronous launch, `p3/STATUS.md`), so asynchrony on Intel needed SYCL kernels on the interop queue (`libalfsb.so`, Section 7.9). The GCC runtime for AMD costs about 150 μs per target region with a parallel loop, against about 15 μs for a HIP launch; AMD needed HIP kernels for all small operations and a patched GCC plugin (Section 10.3). The lesson recorded in `GPU_PLAN_V2.md`: OpenMP target offload is the portable base for bulk kernels, plus a small native kernel layer per vendor behind `alf_la`.

3.3 One source tree, backend by preprocessor

`configure.sh` accepts `GPU=none|nvidia|intel|amd` (`configure.sh:5, 31-36`). For a GPU target it adds `-DALF_GPU` and exactly one of `-DALF_LA_CUDA`, `-DALF_LA_ONEMKL`, `-DALF_LA_HIP`, plus the offload flags of the compiler (Appendix C). `Prog/Makefile:16-24` then adds the seven modules `gpu_state`, `gpu_wrap`, `gpu_lowrank`, `gpu_update`, `gpu_stab`, `gpu_tau`, `gpu_obs`.

The CPU code is changed only by *guarded early branches*: at the start of a CPU routine, inside `#ifdef ALF_GPU`, a test whether the device path is active, a call of the device routine and a `return`. Example, `Wrapgr_mod.F90:141-149`:

```
#ifdef ALF_GPU
   ! GPU port, ticket 205: device-resident G (INTERIM copies per call), host decision
       , same RNG order
   if (gpu_upd_enabled()) then
      if (gpu_upd_selftest_due()) call Wrapgr_gpu_selftest(GR,NTAU,PHASE,...)
      if (gpu_trace_on()) call gpu_trace_begin(NTAU+1)
      call gpu_wrapgrup(GR,NTAU,PHASE,Propose_S0,Nt_sequential_start,
          Nt_sequential_end, N_Global_tau)
      if (gpu_trace_on()) call gpu_trace_end('U',NTAU+1,PHASE)
      return
   endif
#endif
```

(The comment "INTERIM copies per call" is stale: since ticket 208-1 no copy of G is made per call.) Hooks of this kind are in `main.F90`, `Wrapgr_mod.F90`, `tau_m_mod.F90`, `Hamiltonian_Hubbard_smod.F90` (observables), and read-only accessors in `Hop_mod.F90:301-392`, `Operator_mod.F90:1180-1199` and `control_mod.F90:457, 542`. With `GPU=none` none of this code is compiled; the consolidation check found the CPU build bitwise identical to upstream (`CONSOLIDATION.md`).

Code and documents disagree. `RUNTIME_SWITCHES.md` (line 53) names `upgrade_mod.F90` as a hook location; it contains no GPU code. The hooks are in the files listed above.

3.4 Device-resident Green function and stacks

The central decision of the data model (ADR-002, ticket 208-1): the Green function, the UDV stacks and all work arrays live on the device for the whole run. The host keeps a *shadow* copy of G that is brought up to date only when host code reads it (a host observable, the τ pass of the CPU, output). Two logical flags per flavour, `g_dev_ok` and `g_host_ok` (`gpu_state_mod.F90:161-164`), say which copy is current. `g_in` uploads only stale flavours, `g_out` only marks the host copy stale, and `gpu_g_host` downloads on demand (`gpu_update_mod.F90:240-248`).

Code and documents disagree. ADR-002 still states "host objects stay the source of truth". Since ticket 208-1 the device copy of G is authoritative and the host copy is lazily stale.

3.5 Plain arrays instead of ALF's derived types

ALF describes the model through derived types: `Op_T` and `Op_V` arrays of `type(Operator)` with allocatable components, a polymorphic hopping class in `Hop_mod`, and `class(ham_base)` for the Hamiltonian. Such objects cannot be used inside device kernels with the compilers in use: a deep copy of nested allocatables to the device is not portable, polymorphic calls do not exist in device code, and allocation inside kernels is forbidden by the subset.

The port therefore builds, once at setup, a *flat mirror*: integer index arrays and real or complex value arrays that contain everything the kernels need (`gpu_state_mod.F90`, `build_hopping`, `build_groups`, `build_V`, `build_vrows`; Section 4). Hamiltonian objects are read only through accessor routines. Anything that cannot be expressed in the mirror is refused at setup by `gpu_supported`, and the run continues on the CPU (Section 13).

3.6 What "host-assisted" means

Some steps stay on the host on purpose, even in a GPU build (`GPU_PLAN_V2.md`, architecture item A6):

- **The random numbers, the proposal and the Metropolis decision.** The host draws the same random numbers in the same order as the CPU code, so a GPU run follows the same Markov chain as a CPU run of the same compiler and seed. This is the basis of the verification (Section 11). The cost is a transfer of a few numbers per proposal or per block.
- **The sort of the column norms** in the pre-pivoted QR: N numbers go D→H, a permutation of N integers goes back.
- **`ZGEQP3` under `STABLOG`**: no device QR with column pivoting exists, so the matrix goes to the host and back (Section 6).
- **Inside oneMKL**: the default `geqrf` of oneMKL factorises its panels on the host CPU ("hybrid"), and its `getrf` on Intel was measured at 13 ms at $N = 1024$ (`SPEEDUPS.md`, section 4).
- **Small dense algebra**: in the submatrix update the $k \times k$ matrix M ($k \leq N_\mathrm{d}$) and the block $G_0(B, B)$ live on the host (Section 7).
- **Bookkeeping**: counters, binning, MPI reductions and output.

3.7 The library layer `alf_la`

`alf_la` (`Libraries/Modules/alf_la_mod.F90`, 2115 lines) is the only place where vendor libraries are called. Its interface follows LAPACK conventions: explicit-shape arrays with leading dimensions, `complex(8)` throughout, 32-bit pivots, work space owned and grown by the module (`alf_la_mod.F90:38-46`). Arguments are host arrays that have been mapped to the device; inside, each routine obtains the device address with `!$omp target data use_device_addr(...)` and passes `c_loc` of it to a `bind(C)` interface of the library. The module is not thread safe.

The file contains two modules: `alf_la_om_mod` for oneMKL (lines 125-978) and the public `alf_la_mod` (lines 980-2115) with the host and CUDA code inline. For Intel and AMD it forwards to routines named `om_*`; the AMD module `alf_la_hip_mod` exports the same names, so both share one interface (`ALF_LA_OM` is defined for both, lines 104-114). A guard macro, `LA_GUARD`, stops the run if a host code path is reached in a GPU build, so host BLAS can never touch device addresses (lines 117-123).

Table 2 lists every public routine and its implementation per backend.

Table 2: The public routines of `alf_la` (`alf_la_mod.F90:999-1004`) and their implementation. H: `GPU=none`; N: NVIDIA; I: Intel; A: AMD. "dispatch" means `!$omp dispatch` on a oneMKL BLAS routine, which ifx replaces by the device variant.

Routine	Purpose	H	N	I	A
`la_init`	handles, stream, info slots	no-op	cuBLAS/cuSOLVER handles on the OpenACC sync stream; deterministic mode	interop SYCL queue, `alfsh_init`, device identity check	`hipSetDevice`, hipBLAS/hipSOLVER handles
`la_finalize`	free	–	destroy handles	destroy interop	`alfhp_finalize`
`la_sync`	wait, check info	no-op	stream synchronise + slot read	`wait_and_throw` + slot check	presync + `hipDeviceSynchronize`
`la_backend_name`	string	host	cuda	onemkl	hip
`la_reserve_work`	grow work	host	host + device	device	device
`la_gemm`	ZGEMM	`zgemm`	`cublasZgemm_v2`	dispatch `zgemm`	`hipblasZgemm`
`la_gemm_cr`	complex×real as DGEMM on interleaved view	`dgemm`	`cublasDgemm_v2`	dispatch `dgemm`	`hipblasDgemm`
`la_gemm_batched`	strided batch	loop	cuBLAS strided batched	dispatch `zgemm_batch_strided`	hipBLAS strided batched
`la_geru`	rank-1	`zgeru`	`cublasZgeru_v2`	dispatch `zgeru`	`hipblasZgeru`
`la_trsm`	triangular solve	`ztrsm`	`cublasZtrsm_v2`	dispatch `ztrsm`	`hipblasZtrsm`
`la_geqrf(_lwork)`	QR, no pivoting	`zgeqrf`	`cusolverDnZgeqrf`	oneMKL `geqrf` via SYCL shim (hybrid); own device QR with `ALFSH_DEVQR`	own HIP QR ($m \leq 2304$), else hipSOLVER
`la_ungqr(_lwork)`	form Q	`zungqr`	`cusolverDnZungqr`	oneMKL `ungqr`	hipSOLVER
`la_ormqr(_lwork)`	apply Q	`zunmqr`	`cusolverDnZunmqr`	oneMKL `unmqr`; own kernel with `ALFSH_DEVUNMQR=1`	own HIP kernel ($n \leq 2304$, not `C`), else hipSOLVER
`la_getrf(_lwork)`	LU	`zgetrf`	`cusolverDnZgetrf`	oneMKL `getrf`, 64-bit pivots converted on device	`hipsolverZgetrf`
`la_getrs`	LU solve	`zgetrs`	`cusolverDnZgetrs`	oneMKL `getrs`	`hipsolverZgetrs`
`la_qrpp`	column-norm pre-pivot + `la_geqrf`	host norms, sort	device norms (`k_colnorms`), host sort, device permute	same, OpenMP kernels	same, HIP kernels
`la_geqp3_host`	QR with column pivoting (`STABLOG`)	`zgeqp3`	D→H, host `zgeqp3`, H→D	same	same
`la_debug_poison_last_info`	test hook	stop	poison device slot	host slot	host slot

Error handling without synchronisation. cuSOLVER returns its `info` value in device memory. Reading it after every call would force a synchronisation. `alf_la` therefore gives each solver call its own slot in a device array of 4096 integers and reads all slots back in `la_sync`, or when the slots run out (`alf_la_mod.F90:997-998, 1398-1415`). A non-zero `info` then stops the run. `la_getrf` tolerates `info`> 0 (exactly singular matrix), as the CPU code does.

Ordering of library calls and kernels. A library call and an OpenMP kernel are executed in order only if they are submitted to the same *stream* (Appendix A). nvfortran does not implement `!$omp interop`, so the stream of the OpenMP runtime is obtained through the OpenACC runtime, `acc_get_cuda_stream(acc_async_sync)`, which returned `cudaStreamPerThread` with nvfortran 26.9 (`alf_la_mod.F90:48-61`). A separately created stream, the legacy null stream and

the stream of `nowait` regions all failed the ordering test `t_order`. Consequently no `nowait` region is used around `alf_la` calls on NVIDIA. On Intel the SYCL queue of the OpenMP runtime is obtained with `!$omp interop init(prefer_type("sycl"), targetsync: iop)`, and the shim waits after every LAPACK call. On AMD the HIP libraries run on the default stream and synchronise per call, unless the patched GCC plugin defers the synchronisation (Section 10.3).

Reproducibility. Ticket 212 made every reduction in the port run in a fixed order, so that two GPU runs with the same input are bitwise identical. OpenMP device reductions combine partial sums in an unspecified order and had made near-tied pivots flip from run to run (`alf_la_mod.F90:1449-1450`). The column norms are therefore computed by one thread per column with a serial loop; cuSOLVER is put into deterministic mode (`cusolverDnSetDeterministicMode`) and cuBLAS atomics are switched off (lines 1212-1217).

Code and documents disagree. ADR-002 lists only the backends HOST, CUDA and ONEMKL, still names a routine `la_gemm_rc` that does not exist, says that `la_sync` is the only blocking call (the oneMKL shim and every hipSOLVER call block), and does not mention the own QR kernels, `libalfsb.so` or `alf_hipk_mod`. The AMD backend also needed HIP kernels outside `alf_la`, contrary to "adding a backend means implementing `alf_la` only".

3.8 What device code looks like

Two short routines from the code show the style. The first is the wrap of a single-site vertex (`gpu_update_mod.F90:280-298`), the device version of `Op_Wrapup`/`Op_Wrapdo` for $N_{\text{non-zero}} = 1$:

```
Subroutine k_site_wrap(G, ld, nd, n, t, dir, r, eo, nt)
  Integer, intent(in) :: ld, nd, n, t, dir, r, eo, nt
  Complex (Kind=Kind(0.d0)), intent(inout) :: G(ld,1)
  Integer :: j, sp
  Complex (Kind=Kind(0.d0)) :: za, zb
  !$omp target teams distribute parallel do private(sp, za, zb)
  do j = 1, nd
     sp = nint(real(nsigma_d(n,t)))
     za = V_exp(eo + 1 + (dir*sp + nt))
     zb = V_exp(eo + 1 + (-dir*sp + nt))
     if (j == r) then
        G(r,r) = zb * ( za * G(r,r) )
     else
        G(r,j) = za * G(r,j)
        G(j,r) = zb * G(j,r)
     endif
  enddo
end subroutine k_site_wrap
```

The directive turns the loop into a kernel: the N iterations are distributed over teams and threads of the device, each thread handling one j, i.e. one element of row r and one of column r. `G` is a dummy argument; the caller passes `G_d(1,1,nf)`, which is present on the device, so no data move. The field and the exponential table are read from the device mirror (`nsigma_d`, `V_exp`). The work is $2N$ complex multiplications, which takes the device well under a microsecond; the launch costs several microseconds (Table 8). That ratio is the problem the update schemes of Section 7 solve.

The second is `la_gemm` (`alf_la_mod.F90:1505-1533`), shown for the CUDA and host branches:

```
subroutine la_gemm(opa, opb, m, n, k, alpha, A, lda, B, ldb, beta, C, ldc)
  integer, intent(in) :: opa, opb, m, n, k, lda, ldb, ldc
  complex(8), intent(in) :: alpha, beta
  complex(8), target :: A(lda,*), B(ldb,*), C(ldc,*)
```

```
    if (m <= 0 .or. n <= 0) return
#if defined(ALF_LA_OM)
    call om_gemm(opa, opb, m, n, k, alpha, A, lda, B, ldb, beta, C, ldc)
#elif defined(ALF_LA_CUDA)
    cab(1) = alpha; cab(2) = beta
    call pre_lib()
    !$omp target data use_device_addr(A, B, C)
    call chk(cublasZgemm_v2(hblas, opa, opb, m, n, k, c_loc(cab(1)), c_loc(A(1,1)),
        lda, &
         c_loc(B(1,1)), ldb, c_loc(cab(2)), c_loc(C(1,1)), ldc), 'zgemm')
    !$omp end target data
    call post_lib()
#else
    LA_GUARD
    call zgemm(opc(opa), opc(opb), m, n, k, alpha, A, lda, B, ldb, beta, C, ldc)
#endif
  end subroutine la_gemm
```

(Simplified from the source, which nests the `#if` blocks and contains `#line` directives.) `use_device_addr` replaces, inside the block, the host addresses of `A`, `B`, `C` by their device addresses, which `c_loc` passes to cuBLAS. `cab` is a 16-byte aligned module array for α and β (cuBLAS crashed with 8-byte alignment). `pre_lib`/`post_lib` are empty unless `ALF_LA_SYNC_ALWAYS` is set. The call returns before the product is finished; ordering with later OpenMP kernels comes from the shared stream.

4 Data model

4.1 The sweep on the device: who does what

Table 3 maps each step of the CPU sweep of `main.F90` to its device replacement, and Figure 1 shows where the data live.

Table 3: CPU routines and their device replacements in one sweep (production path).

CPU (upstream)	Device routine	Module	Host part
initial `WRAPUL` + `udvst`	`gpu_stab_wrapul`, `gpu_stab_store_l`	`gpu_stab_mod`	column-norm sort
initial `CGR`	`gpu_stab_cgr` (lazy)	`gpu_stab_mod`	sort, phase scalar
`WRAPGRUP`	`gpu_wrapgrup`	`gpu_update_mod`, `gpu_lowrank_mod`	proposals, ratios, decisions, M, RNG
`Hop_mod_mmthr`, `mmthl_m1`	`gpu_mmthr`, `gpu_mmthl_m1`	`gpu_wrap_mod`	–
`Op_Wrapup`	block scalings in `blk_end`	`gpu_lowrank_mod`	factors recorded on host
`Upgrade2` / low-rank update	block mode	`gpu_lowrank_mod`	all per-proposal work
`WRAPUR`	`gpu_stab_wrapur`	`gpu_stab_mod`	sort
`udvl=udvst`, `udvst=udvr`	`gpu_stab_exchange`	`gpu_stab_mod`	–
`CGR`, `Control_PrecisionG`	`gpu_stab_cgr`	`gpu_stab_mod`	sort, threshold test, phase product
`Hop_mod_Symm` + `Obser`	`gpu_obs_measure`	`gpu_obs_mod`	counters, scalar accumulation
`WRAPGRDO`	`gpu_wrapgrdo`	`gpu_update_mod`, `gpu_lowrank_mod`	as `WRAPGRUP`

CPU (upstream)	Device routine	Module	Host part
`WRAPUL`	`gpu_stab_wrapul`	`gpu_stab_mod`	sort
`TAU_M` (`Ltau=1`)	`gpu_tau_pass`	`gpu_tau_mod`, `gpu_obs_mod`	scalars
`Pr_obs` per bin	`gpu_obs_flush`, then `Pr_obs`	`gpu_obs_mod`	binning, MPI, I/O

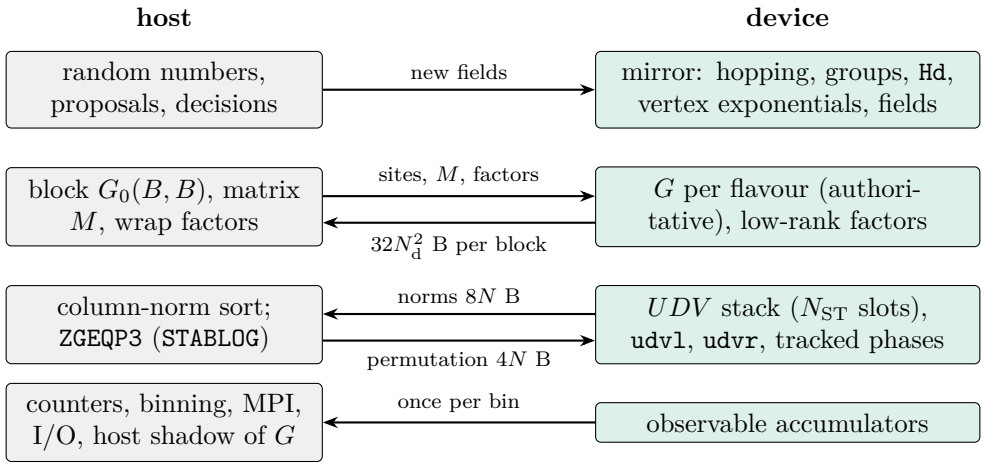


Figure 1: Where the data live during a sweep, and what crosses the bus (production path). Grey: host; green: device.

4.2 How residency is expressed

All device data are Fortran module allocatables with a host twin of the same name. Residency is expressed with OpenMP data directives only (`gpu_state_mod.F90:1165-1206`):

- `!$omp target enter data map(to: ...)` once at setup for the constant mirror (the model), which copies the host values to newly allocated device memory;
- `!$omp target enter data map(alloc: ...)` for work arrays, which allocates device memory without a copy;
- `!$omp target update to(...)` and `from(...)`, often on array sections, for the transfers during the run;
- `!$omp target exit data map(delete: ...)` at the end.

Kernels are `!$omp target teams distribute parallel do` loops over these arrays; the OpenMP runtime finds the device copy because the array is present. `alf_la` obtains device addresses with `use_device_addr`. There is no `is_device_ptr`, `omp_target_alloc` or unified memory anywhere in `Prog/gpu_*.F90`. Arrays are passed to kernels as explicit-shape dummy arguments: ifx copies the descriptors of module allocatables that are referenced inside a target region at every launch (16.0 against 10.7 μs per launch, `gpu_lowrank_mod.F90:1364-1366`).

4.3 The constant mirror of the model

Built on the host in `gpu_state_setup` (`gpu_state_mod.F90:620-703`) and uploaded once. Table 4 lists it.

Table 4: The constant device mirror (`gpu_state_mod.F90:40-97, 146-172`). n_{el}: number of active hopping elements; n_d: number of sites of an element; N_{op}: number of vertices per slice.

Array	Shape, type	Content
`hop_start`	$N_{\mathrm{FL}}+1$, int	first element of each flavour
`hop_meta`	$(4, n_{\mathrm{el}})$, int	kind (1 real, 2 complex), n_d, offset into site list, offset into matrices
`hop_P`	$\sum n_d$, int	site list P of each element
`hop_Rval`	$\sum 4n_d^2$, real(8)	for real elements: $e^{-\Delta\tau T_e}$, its inverse, and the half steps for `Symm` (`mat`, `invmat`, `mat_1D2`, `invmat_1D2`)
`hop_Cval`	$\sum 4n_d^2$ + td part, complex	the same for complex elements, followed by one matrix per (time-dependent element, slice, kind)
`hop_gptr`	$n_{\mathrm{groups}}+1+n_{\mathrm{td}}$, int	checkerboard groups: start of each group in the element list
`hop_aux`	$(2, n_{\mathrm{el}})$, int	offset into the dense copies, flag "P is the identity"
`hop_At`, `hop_Ac`	$4n_d^2$ per dense element	transposed real and complex copies of dense ($n_d > 8$) elements, for the left-multiply variants
`V_meta`	$(4, N_{\mathrm{op}}, N_{\mathrm{FL}})$, int	N of the vertex, offset into sites, offset into exponentials, skip flag ($\lvert g\rvert < \epsilon$)
`V_type`	N_{op}, int	field type (1 Ising, 2 four-valued)
`V_P`	$\sum N$, int	sites of each vertex
`V_exp`	$\sum N(2t+1)$, complex	the table `E_exp`(i, s), $s = -t \dots t$, of the exponentials $e^{g\eta(s)V}$
`vr_ptr`, `vr_ent`	$(N+1, N_{\mathrm{FL}})$, $(2, \cdot)$	for each site r, the list of (vertex, position) pairs in ALF's operator order; used by the fused slice kernels
`nsigma_d`	$(N_{\mathrm{op}}, L_{\mathrm{trot}})$, complex	the auxiliary field; kept current during the sweep
`nsigma_t`	N_{op}, int	field type per vertex

Three details matter for exact agreement with the CPU:

- For real elements the lower triangle is filled from the upper one, as ALF's symmetric multiply routines read only the upper triangle; real elements with $n_d > 8$ keep the full stored matrix, because ALF's `ZLARCM` uses all of it (`gpu_state_mod.F90:765-778`).
- For complex elements with $n_d > 8$ the imaginary part of the diagonal is set to zero, as `ZHEMM` ignores it (lines 779-786).
- Inactive elements ($\lvert g\rvert^2 \le$ `Zero`) are not stored, as the host skips them.

Host-only tables: `hop_gstart`, `hop_big` and, for time-dependent hopping, `td_grp`$(L_{\mathrm{trot}}, 2, n_{\mathrm{td,max}}, N_{\mathrm{FL}})$ and `td_n`. These select which group a kernel launch processes and need not be on the device.

Code and documents disagree. ADR-002 (section 2) describes arrays `cb_idx`, `cb_exp` (bond pairs and 2×2 exponentials), `T_exp`$(N, N, F, 4)$, `V_site` and `rnd_d`. None exists. The code uses a general element list with blocks of any size n_d, checkerboard groups and dense copies, and time-dependent hopping, which the ADR does not cover.

4.4 Work arrays

Table 5: Device work arrays. N_{ST}: stabilisation slots (`NSTM`); F: N_{FL}; F_e: $N_{\mathrm{FL,eff}}$. All mapped with `map(alloc:)`.

Array (module)	Shape	Mirrors / role
`G_d` (state)	(N, N, F)	`GR`; authoritative copy of G
`udv_U`, `udv_V` (state)	$(N, N, N_{\mathrm{ST}}, F_e)$	the stack `udvst`: U and V of each stabilisation point

Array (module)	Shape	Mirrors / role
udv_D / udv_L	$(N, N_{\mathrm{ST}}, F_e)$	scales: complex D (default) or real $\log D$ (STABLOG); the other is a $1 \times 1 \times 1$ dummy
udw_U, udw_V	$(N, N, 2, F_e)$	slot 1 = udvl, slot 2 = udvr
udw_D / udw_L	$(N, 2, F_e)$	their scales
GT0_d, G0T_d, GTT_d	(N, N, F)	only with Ltau=1: $G(\tau,0)$, $G(0,\tau)$, $G(\tau,\tau)$
scr_c, scr_vec, scr_r, scr_ipiv, scr_work	$(N, N, 2)$, $(N, 4)$, $(N, 2)$, N, $64N + 64$	scratch; scr_vec(:,1:2) holds the rank-one vectors
Hd (wrap)	$(N, N, F, 4)$	precomputed dense hopping products, only for $N \le 800$ (Section 5)
sA, sB, sC, st_Gn, st_Gt (stab)	(N, N) each	stabilisation scratch, new and wrapped G
st_tau, st_Dl, st_nrm, st_ipvt, ...	N	Householder scalars, scales, norms, pivots
st_dw, st_dv	$(2, F_e)$, (N_{ST}, F_e)	tracked phases $\det U/\lvert\det U\rvert$ of the work slots and the stack
t_H1, t_H2, t_Q, t_A, t_B (tau)	$(2N, 2N)$	CGR2_2 work; Q, A, B only with the Newton–Schulz step
G00_d, t_GR, t_old, t_S (tau)	(N, N, F), (N, N, F), $(N, N, 3)$, $(N, N, 4)$	$G(0,0)$, reference copy, previous values for the precision check
X_d, Y_d (lowrank)	(N, N_{d}, F)	low-rank factors; Y_d only for the delayed scheme
gbd, cbuf, bsd, bS_d (lowrank)	$(N_{\mathrm{d}}, N_{\mathrm{d}}, 2)$, $(2N_{\mathrm{d}} + N_{\mathrm{d}}^2) \cdot 2$, ...	block-mode buffers (Section 7)
acc, acc0, tacc, tacc0, lacc1, lacc2 (obs)	per observable	accumulators of the device observables

4.5 Memory footprint

Summing the shapes (complex numbers are 16 bytes) gives, per run,

$$M \approx 16N^2 \Big[\underbrace{4N_{\mathrm{ST}}}_{\substack{\text{stack } U,V \\ (F_e=2)}} + \underbrace{2}_{G} + \underbrace{8}_{\text{udw}} + \underbrace{2}_{\text{scr}} + \underbrace{5}_{\text{stab}} + \underbrace{16}_{\text{dense hop}} + \underbrace{8}_{\texttt{Hd}} \Big] + O(NN_{\mathrm{ST}}). \tag{11}$$

The dense hopping term is hop_Rval + hop_At + hop_Ac, about $128N^2$ bytes per flavour. The bracket without the stack is 41, which agrees with the empirical rule of TUNING_DEFECT.md,

$$M_{\mathrm{peak}} \approx (4N_{\mathrm{ST}} + 40) \cdot 16N^2 \text{ bytes} + 0.3 \text{ GB (CUDA context)}, \tag{12}$$

which was checked at $L = 48$, $\beta = 48$ (19.7 GB predicted, 19.6 GB measured) and $L = 32$ (2.8 predicted, 3.1 measured). With Ltau=1 the τ pass adds about $480N^2 + 32N^2F$ bytes. The stack dominates: it grows as $N^2 L_{\mathrm{trot}}/\mathtt{Nwrap} \propto L^5/\mathtt{Nwrap}$ at $\beta \propto L$. This is why Nwrap=5 costs 45 % more memory than Nwrap=10 at $L = 24$ (1553 against 1069 MiB, TUNING_DEFECT.md). The host does not mirror the stack; the resident set of the host process at $L = 36$ was 1.9 GB while the device held 5.2 GB (TUNING_DEFECT.md).

The code prints the size of every gpu_state array and their total into the info file (report_layout, gpu_state_mod.F90:1223-1271). That total does not include the arrays of the other modules.

4.6 All host–device transfers in one sweep

Table 6 is the complete list for the production path (device updates in submatrix block mode, device stabilisation, device observables, Ltau=0), assembled from the code. Sizes are in bytes.

Table 6: Host–device transfers per sweep, production path. $b \le N_\mathrm{d}$: vertices in a block; k: pending sites at a flush.

When	Direction	Size	What and where
per proposal	–	0	nothing; decided from the host block (`gpu_lowrank_mod.F90:1693-1720`)
per block, begin	H→D	$8N_\mathrm{d}$	block sites `bsd`
per block, begin	D→H	$32N_\mathrm{d}^2$	block $G_0(B,B)$ for 2 flavours, `gbd`; 32 KB at $N_\mathrm{d} = 32$
per block, end	H→D	$32(2N_\mathrm{d} + N_\mathrm{d}^2)$	`cbuf`: wrap factors and packed M; 34.8 KB at $N_\mathrm{d} = 32$
per block, end	H→D	$8N_\mathrm{d}$, $16b$	pending sites `bS_d`; new fields `nsigma_d(vmin:vmax,t)`
per slice	–	0	G is not moved (`g_in` is a no-op, `g_out` sets a flag)
per decompose (each flavour)	D→H, H→D	$8N$, $4N$	column norms, permutation (`la_qrpp`)
per decompose	D→H	24	scale test (`dev_testscale`)
per `CGR`	D→H, H→D	$8N$, $4N$	`la_qrpp` of the middle matrix
per `CGR`	D→H	16 + 32	phase; precision numbers (max, sum, two NaN counts)
per `CGR` (`STABLOG`)	D→H, H→D	$16N^2$, $16N^2 + 20N$	`la_geqp3_host`
per measurement	D→H	3 reductions	kinetic, potential energy, density
per bin	D→H, H→D	size of accumulators	`gpu_obs_flush`: download, zero, upload
amortised	D→H	4× slots	`info` slots of `alf_la`

At $L = 24$ and $N_\mathrm{d} = 32$ there are 8 640 blocks per sweep (18 per slice × 480 slices) and 43 200 copies, four H→D and one D→H per block (`p7/NOTES.md`, line 94). Before block mode, the per-proposal transfers dominated: on the A30 at $N = 400$ there were 432 000 stream synchronisations and 46 000 small D→H copies per run of six sweeps, and the kernels ran only 1.4 s out of 24 s (`DELAYED_SUBMATRIX.md`).

Other paths add transfers: a host observable needs $16N^2$ per flavour and measured slice (`gpu_g_host`); the standard rank-one update needs 16 bytes of G_{pp} per proposal and flavour and 16 bytes of field per acceptance; the τ pass with host observables needs $3 \cdot 16N^2$ per slice and flavour. With `Ltau=1`, `main.F90:1301` downloads G once before every τ pass even when the pass runs entirely on the device.

Code and documents disagree. The info line of `gpu_update_mod.F90:109` still says "INTERIM per-slice G copies"; lines 52-53 of the same file and the code say that G is device-authoritative and never copied per slice.

5 Propagation on the device

The propagation routines are in `Prog/gpu_wrap_mod.F90` (2660 lines). They reproduce the routines of `Hop_mod.F90` and `Operator_mod.F90` on device matrices, in the same order of factors, so that the device result agrees with the CPU to rounding.

5.1 Hopping: five routines, three code paths

ALF applies $e^{\pm\Delta\tau T}$ through five routines that differ in side (left or right multiplication), matrix (exponential or inverse) and the order of the elements. The device versions are listed in Table 7.

Table 7: Device hopping routines (`gpu_wrap_mod.F90:117-173`). "left" acts on the rows of M, "right" on its columns. Matrix index k: 1 $e^{-\Delta\tau T}$, 2 inverse, 3 and 4 half steps for `Symm`.

Device routine	ALF routine	Side	k	Element order
`gpu_mmthr`	`Hop_mod_mmthr`	left	1	descending
`gpu_mmthr_m1`	`Hop_mod_mmthr_m1`	left	2	ascending
`gpu_mmthl`	`Hop_mod_mmthl`	right	1	ascending
`gpu_mmthlc`	`Hop_mod_mmthlc`	left	1	ascending
`gpu_mmthl_m1`	`Hop_mod_mmthl_m1`	right	2	descending

(a) Dense precomputed product (`Hd`). For $N \le$ `HOPD_NMAX` $= 800$ (`gpu_wrap_mod.F90:104`) each of the four hopping sequences used in the sweep is applied once at setup to the identity. The result is a dense $N \times N$ matrix per flavour and sequence, `Hd`$(N, N, N_{\mathrm{FL}}, 4)$ (`hopd_build`, lines 262-285). Applying a sequence is then one `la_gemm` into scratch plus a copy kernel (lines 287-303). The comment at lines 252-254: "applying one is a GEMM plus a copy instead of one launch per checkerboard group. Used for Ndim $\le$ HOPD_NMAX; above that the group kernels are cheaper." This path is used regardless of the `Checkerboard` flag. For the production model the static hopping is dense anyway. `gpu_mmthlc` has no `Hd` variant, so `WRAPUL` uses the group kernels.

Mathematically `Hd` $= \prod_e B_e$ is the same matrix as the product of the elements; numerically it differs from applying the elements one by one in the rounding of the product, at the level of 10^{-16} relative per application.

(b) Checkerboard groups. When all elements have $n_d \le 8$ sites, consecutive elements whose site lists are pairwise disjoint are combined into a *group* (`gpu_state_mod.F90:66-73`). Elements of a group commute, so one kernel launch can apply all of them in parallel. For the square lattice with nearest-neighbour hopping there are four groups, so one B_T costs four launches. The kernels are `k_cb_left_g` and `k_cb_right_g` (lines 638-669):

- `k_cb_left_g`: `target teams distribute parallel do collapse(2)` over (column tile, element). Each thread handles `gpu_tl` consecutive columns (default 1) and computes, for its element e and column j, the n_d new values $M_{P_e(i),j} \leftarrow \sum_l (B_e)_{il} M_{P_e(l),j}$.
- `k_cb_right_g`: the same over (element, row tile), `gpu_tr`=2 rows per thread by default, "tuned on the A30".

The inner sums are written out by hand for $n_d = 1$ (a scale), $n_d = 2$ (an explicit 2×2) and general $n_d \le 8$ (the element is gathered into a local array `w(8)`), in the order $l = 1 \dots n_d$ of ALF's `ZDSLSYMM` and `ZSLHEMM` (`el_left_d`, `el_right_d`, lines 479-633). Real elements are applied with real arithmetic on the real and imaginary parts of M separately, as ALF does.

A variant that applied all groups in one launch with a barrier between groups was removed in ticket 205: "slower on the A30 and wrong on the Arc B580" (lines 47-51).

(c) Dense elements ($n_d > 8$). If the site list is not the identity, the rows (or columns) are gathered into scratch, multiplied, and scattered back (`dense_left`, `dense_right`, lines 674-741). Complex elements use `la_gemm`. For a real element times a complex matrix ALF uses `ZLARCM`; the device uses `la_gemm_cr`, which views the complex matrix as a real matrix of twice the height

and calls one DGEMM:

$$MB \text{ with } B \text{ real}: \quad \begin{pmatrix} \Re M \\ \Im M \end{pmatrix}_{\text{interleaved}} B, \tag{13}$$

which costs half the flops of a ZGEMM. For a left multiplication BM the interleaving does not work directly. Two modes exist (`gpu_dense_left_mode`, line 95; automatic: mode 1 for $n_d \leq 400$, mode 2 above, line 687): mode 1 multiplies with a complex copy `hop_Ac` (four times the flops), mode 2 transposes, uses $(BM)^T = M^T B^T$ with `hop_At` and `la_gemm_cr`, and transposes back.

5.2 Time-dependent hopping (the point defect)

The point defect is a hopping element whose coupling depends on the slice, $g_t(\tau)$ (`TIMEDEP_HOPPING.md`). The element is $A(g) = Ue^{gE}U^\dagger$. Instead of uploading a new matrix per slice, setup appends ordinary complex elements to the mirror, one per active slice and kind (`gpu_wrap_mod.F90:117-120, 176-212`):

- propagation: $A(g_t)$ as $k = 1$ and $A(-g_t)$ as $k = 2$;
- `Symm`: $A(\bar{g}/2)$ and $A(-\bar{g}/2)$ as $k = 3, 4$, with $\bar{g}$ the average of $g_t(\tau)$ and $g_t(\tau - 1)$.

Each sits in a group of its own. The host table `td_grp`$(\tau, \text{kind}, j, f)$ names the group to launch, 0 meaning inactive. Since E has a single non-zero eigenvalue for the defect, the support reduction `td_reduce` stores only the rows where eigenvectors with $E_k \neq 0$ are non-zero (relative threshold 10^{-13}): one site. Without it the set of matrices at $L = 32$ would need more than 2 GB; `gpu_supported` refuses sets above 2 GB (`gpu_state_mod.F90:601-602`).

Because time-dependent elements are the last of their flavour (`gpu_supported` requires this), they are applied first in the descending routines and last in the ascending ones, around the static `Hd` product.

5.3 Interaction vertices

Each vertex $\text{Op}_V(n)$ of the Hubbard model acts on one site p_n. Its exponential for field value s is the diagonal factor `E_exp`$(1, s)$, stored in `V_exp`. The field is read on the device as $s = \text{nint}(\Re\,\texttt{nsigma_d}(n, \tau))$.

- `gpu_Vop_R`, `gpu_Vop_L` (lines 856-917): `Op_mmultR` and `Op_mmultL` for one vertex: scale rows (or columns) $P(i)$.
- `gpu_Vop_wrap` (lines 921-963): `Op_Wrapup`/`Op_Wrapdo`: rows by $E(i, \pm s)$, then columns by $E(i, \mp s)$.
- `k_vslice_R`, `k_vslice_L` (lines 980-1124): all vertices of a slice in one launch. Threads run over (column block of `gpu_vb`=8 columns, row r). For row r all vertices containing r are applied in ALF's order through `vr_ptr`/`vr_ent`. Up to four factors per row are precomputed per thread. "the same product sequence as the loop of calls" (lines 966-968).
- `k_vwrap_all` (lines 1137-1190): the whole-slice wrap in one launch; row and column factors are merged in operator order.
- `k_site_wrap` (`gpu_update_mod.F90:280-298`): the single-site wrap used inside the update loop:

$$G_{rj} \leftarrow z_a G_{rj},\ G_{jr} \leftarrow z_b G_{jr}\ (j \neq r), \qquad G_{rr} \leftarrow z_b(z_a G_{rr}), \tag{14}$$

  with $z_a = \texttt{V_exp}(\cdot, \text{dir} \cdot s)$ and $z_b = \texttt{V_exp}(\cdot, -\text{dir} \cdot s)$. The diagonal is scaled row first, then column, "ALF's `ZSCAL` order".

In the sweep the wrap of G cannot use the fused slice kernels: wraps and updates alternate vertex by vertex. In the delayed and submatrix schemes the per-vertex wrap is not applied to the device G at all but folded into the low-rank representation (Section 7).

5.4 Composite operations

- In the sweep (`gpu_update_mod.F90:470-629`): `gpu_wrapgrup` applies `gpu_mmthr` and `gpu_mmthl_m1` to G, i.e. $G \leftarrow e^{-\Delta\tau T} G e^{\Delta\tau T}$, then loops over the vertices with wrap and update. `gpu_wrapgrdo` does the updates first, then `gpu_mmthl` and `gpu_mmthr_m1`.
- For the stabilisation (`gpu_wrap_mod.F90:1196-1217`): `gpu_wrapur_slices` applies, for $\tau = \tau_1 + 1 \ldots \tau_2$, `gpu_mmthr` then `gpu_Vslice_R`, i.e. $U \leftarrow B_\tau U$. `gpu_wrapul_slices` applies the conjugate factors in reverse order for the left side.
- `Symm` (`gpu_symm`, lines 217-249): groups in descending order, half steps $k = 3$ from the left and $k = 4$ from the right. For the measurements `gpu_symm_build` (lines 360-390) forms the static matrices A and B once, so that $\mathrm{Symm}(G) = AGB$ is two GEMMs; `gpu_symm_td` adds the time-dependent half steps, $A(A_{td} G B_{td})B$.

The flavour loop is on the host: `do nfe=1,N_FL_eff; nf=Calc_Fl_map(nfe)`. There is no batching over flavours.

Code and documents disagree. The header of `gpu_wrap_mod.F90` (line 37) says "Nothing in the sweep calls these yet"; the routines are called by the update, stabilisation, τ and observable modules. `HARD.md` plans "one kernel per checkerboard family" and "both flavours per launch"; the code launches per group, loops over flavours on the host, and uses the dense `Hd` product for $N \leq 800$ even with checkerboard hopping.

5.5 Operation counts

The following counts are derived from the code, not measured. Per slice and flavour:

- `Hd` path, $G \leftarrow BGB^{-1}$: 2 ZGEMMs of size N, $2 \times 8N^3$ real flops, plus two copies of N^2. At $N = 576$ this is 3.1 GFlop per slice and flavour, or about 2.9 TFlop per sweep ($2L_{\mathrm{trot}} = 480$ slices, 2 flavours). At the FP64 peak of an A30 (about 10 TFlop/s with tensor cores according to the vendor data sheet, not measured here) that alone is at least 0.3 s of the 5.25 s sweep.
- Checkerboard path, square lattice: about $2N$ bonds, each 2×2 on N columns, about $24N^2$ flops per B_T in four launches.
- Vertex wraps: $N_{\mathrm{op}} = N$ single-site launches of $2N$ complex multiplications each, i.e. $O(N^2)$ work in N launches. These launches are the reason for the block mode of Section 7.
- `WRAPUR` per slice: 1 ZGEMM (or 4 group launches) plus 1 fused vertex launch.

The kernel benchmark of Phase 1 on the A30 gives the scale (`BENCHMARKS.md`): a dense B_T product takes 53, 172 and 2757 μs at $N = 144$, 400, 1024 on the device against 1095, 22 941 and 388 561 μs on one host core; the vertex wrap with one launch per site took 1621 μs at $N = 144$ (host 199 μs), and fused into one launch 13 μs (host 36 μs). The last pair shows the problem of launch latency in one number: a fused kernel is 125 times faster than one launch per site.

6 Stabilisation on the device

The stabilisation is in `Prog/gpu_stab_mod.F90` (2118 lines). It replaces `WRAPUR`, `WRAPUL`, the stack handling of `main.F90`, `CGR` and `Control_PrecisionG`. On the device the full sequence runs without moving any $N \times N$ matrix to the host (in the default stabilisation).

6.1 Stacks and slots

The stack `udvst` of ALF becomes `udv_U`, `udv_V`, `udv_D` (or `udv_L` for `STABLOG`) with one slot per stabilisation point and flavour. The two working decompositions `udvl` and `udvr` are slots 1 and 2 of `udw_*`. The exchanges of `main.F90` ("`udvl = udvst(NST); udvst(NST) = udvr`" on the way up, the mirror image on the way down) are device-to-device copies (`gpu_stab_exchange`, `gpu_`

stab_mod.F90:350-414). The tracked phases of $\det U$ (st_dw, st_dv) travel with the slots. Only the CPU τ pass needs the stack on the host; then udvst is downloaded on the way down.

The calls in main.F90 (lines 706-1294) follow the CPU code one to one: gpu_stab_wrapul with gpu_stab_store_l builds the initial left stack, gpu_stab_cgr computes the initial G, and in the sweep gpu_stab_wrapur/wrapul, gpu_stab_exchange and gpu_stab_cgr replace their CPU counterparts at each stabilisation point. Op_phase, the phase product and Control_PrecisionP stay on the host; they need only scalars.

6.2 The QR decomposition with pre-pivoting

The problem. ALF factorises with ZGEQP3, the QR decomposition with column pivoting. At step j it chooses, among the remaining columns, the one with the largest norm *of its component orthogonal to the columns already chosen*, and updates those partial norms after each step. That makes every step depend on the previous one and on a global maximum: it is a sequence of matrix–vector operations and cannot be blocked into matrix products. No GPU library offers it: cuSOLVER and oneMKL have none, and MAGMA's zgeqp3 is bound by matrix–vector work and runs only on NVIDIA and AMD (HARD.md, section 2). Running ZGEQP3 on the host would cost a transfer of the matrix both ways and the host time: about 5.7 s per CGR2_2 at $N = 1024$, and 14 ms at $N = 400$ and 186 ms at $N = 1024$ for one ZGEQP3 on one core (HARD.md; ADR-001, item 4).

The solution: pre-pivoting. la_qrpp (alf_la_mod.F90:1974-2065) sorts the columns once, by their initial norms, in descending order, and then calls the unpivoted, blocked la_geqrf:

1. k_colnorms: the scaled 2-norm of every column, one device thread per column with a serial loop (fixed summation order, ticket 212);
2. D→H of the N norms; check for NaN or Inf;
3. on the host, a stable insertion sort in descending order, giving a permutation with the semantics of ZGEQP3's jpvt;
4. H→D of the permutation; k_permute_cols;
5. la_geqrf.

Why this suffices for ALF: before the decomposition the columns of BU are multiplied by the scales D of the previous decomposition (k_scale_cols). These scales span many orders of magnitude, so the initial column norms are already strongly graded, and the order that ZGEQP3 would choose differs from the order of the initial norms only among columns of similar size. The scales $|R_{ii}|$ then come out in nearly descending order, which is what the stabilisation needs.

The comparison test. Ticket 104 compared, on the CPU, ALF with ZGEQP3 and ALF with pre-pivoting plus ZGEQRF, at the same seeds. For the default stabilisation the mean precision of G stayed within $\pm 5\,\%$ and the Monte Carlo trajectories were identical (HARD.md, section 2). For STABLOG the maximum deviation of G became up to 14 times larger ($4.4 \times 10^{-6} \to 6.2 \times 10^{-5}$ at $L = 8$, $\beta = 20$, $U = 4$), with more "Unstable matrix" aborts at $U = 8$. Ticket 108 tried a two-pass sort and a blocked re-ranking; both failed (outliers up to 4.6 times in the maximum and about 1.4 times higher means). Therefore STABLOG keeps the host ZGEQP3 in CGR (la_geqp3_host). The unit test t_qrpp compares la_qrpp with QDRP_decompose called with all columns fixed after the same sort; the device and host norms can differ in the last bit, so columns with tied norms may be ordered differently, "an equally valid pivot order" (alf_la_mod.F90:1980). The CPU build (GPU=none) keeps ZGEQP3 (owner decision at the Phase 1 gate, PLAN.md, section 5).

6.3 Decompose (default stabilisation)

dev_decompose (gpu_stab_mod.F90:489-583) computes UDV from $\tilde{U} = B(\tau_2, \tau_1)U_{\text{old}}$ (already propagated) in these steps:

1. `k_scale_cols`: $\tilde{U}_{:,j} \leftarrow \tilde{U}_{:,j}\, D_j$.
2. `la_qrpp`: $\tilde{U}P = QR$, Householder scalars τ_i in `st_tau`.
3. `k_diagabs`, `k_divrows`: $D_i = |R_{ii}|$, $R_{ij} \leftarrow R_{ij}/|R_{ii}|$ for $j \geq i$ (so $R = D\tilde{R}$).
4. `Pivot_phase` on the host: the sign of the permutation (from the pivot vector, no transfer of matrices).
5. `k_phase`: one serial device loop computing the phase of $\prod_i \tilde{R}_{ii}$, and the phase of $\det Q$ from the τ_i (see below).
6. `k_scale_row1`: the first row of $\tilde{R}$ is divided by that phase; "to set Det(R)=1 [=Det(V)]" as the CPU code says (`udv_state_mod.F90:527`); step 10 multiplies the first column of U by it, as the CPU does with `ZSCAL` (line 614).
7. Permute the old V with the pivot vector (`ZLAPMR`/`ZLAPMT` equivalents), and form $V \leftarrow \tilde{R}V$ (right side) or $V \leftarrow V\tilde{R}^\dagger$ (left side) with `la_gemm`; the triangular factor is multiplied as a full matrix.
8. Form Q by applying it to the identity with `la_ormqr` (not `la_ungqr`, see below), copy to U.
9. Newton–Schulz step on U (below).
10. `k_scale_col1`: $U_{:,1} \leftarrow U_{:,1} \cdot \text{phase}$.
11. `dev_testscale`: one D→H of three numbers; a NaN scale is an error ("Switch to LOG is required"), $D_1 > 0.1\,$`huge` or $D_N < 10\,$`tiny` gives a warning.

The cost is about $37N^3$ real flops per decompose (`geqrf` $\frac{16}{3}N^3$, `ormqr` on I $8N^3$, $\tilde{R}V$ $8N^3$, Newton–Schulz $16N^3$; derived from the code).

Why `ormqr` on the identity and a Newton–Schulz step. With `la_ungqr` the precision of G in the sweep was 4 times (mean) and 11 times (max) worse than on the CPU at $N = 36$; applying Q to I with `ZUNMQR` restored the CPU level at $N = 36$, "at N = 144 the Newton–Schulz step below is needed in addition" (`gpu_stab_mod.F90:559-568`). The cause is that cuSOLVER's Q is less orthogonal than LAPACK's: $\|Q^\dagger Q - I\|$ about 1.6×10^{-14} against 3.3×10^{-15} at $N = 400$ (`DISCREPANCIES.md`). One Newton–Schulz step,

$$Q \leftarrow Q\,\tfrac{1}{2}(3I - Q^\dagger Q), \tag{15}$$

reduces the orthogonality defect quadratically: with $Q^\dagger Q = I + \Delta$, the new Q has $Q^\dagger Q = (I - \frac{1}{2}\Delta)(I + \Delta)(I - \frac{1}{2}\Delta) = I - \frac{3}{4}\Delta^2 + O(\Delta^3)$. It costs two GEMMs (`sA`$= U^\dagger U$, kernel `k_ns` forming $-\frac{1}{2}$`sA`$+\frac{3}{2}I$, `sB`$= U$ `sA`). Measured in ticket 206 (12×12, $\beta = 4$, same seeds): without the step, precision of G mean 9.7×10^{-11} and max 1.2×10^{-6}; with it 5.4×10^{-11} and 3.8×10^{-7}; CPU 4.1×10^{-11} and 7.2×10^{-7} (`gpu_stab_mod.F90:565-568`). The step is on by default for all backends; `ALF_GPU_NS=0` switches it off (debug only). oneMKL's Q does not need it, but it was kept for uniformity (`DECISIONS.md`, item 26).

6.4 Decompose with `STABLOG`

With `STABLOG` the scales are kept as $\log D$ (`gpu_stab_mod.F90:515-544`). `k_lognorm` computes $\log\|\tilde{U}_{:,j}\|_2 + L_j$ for every column, the host sorts with the CPU's selection sort (`sort_stablog`, the same one-sided swap `tn(pvt)=tn(i)` as the CPU code), the permutation goes back, U, V and L are permuted on the device, and an unpivoted `la_geqrf` follows. `k_stablog_scale` then applies $U_{ij} \leftarrow U_{ij}\, e^{L_j - L_i}$ for $j > i$ and $L_i \leftarrow \log|R_{ii}| + L_i$. Large and small scales exist only in logarithmic form.

6.5 Assembling G on the device (`CGR`)

`dev_cgr` and `cgr_solve` (`gpu_stab_mod.F90:869-958`) evaluate Eq. (7):

$$\texttt{sA} = U_R^\dagger U_L, \qquad \texttt{sB} = V_R V_L, \tag{16}$$

$$T = D_R\,(V_R V_L)\,D_L + U_R^\dagger U_L \qquad (\texttt{k_tpup} : T_{ij} = D_{R,i} T_{ij} D_{L,j} + \texttt{sA}_{ij}), \tag{17}$$

$$G = U_L\,T^{-1}\,U_R^\dagger. \tag{18}$$

With `STABLOG`, `k_tpup_log` implements the four sign cases of `cgr1_mod.F90`, so that no exponential overflows; for example $T_{ij} = \texttt{sA}_{ij} + e^{L_{R,i}+L_{L,j}} T_{ij}$ when both logarithms are ≤ 0 and $T_{ij} = e^{-L_{R,i}-L_{L,j}}\texttt{sA}_{ij} + T_{ij}$ when both are positive.

T^{-1} is applied through a QR of T with pre-pivoting, $TP = QR = QD\tilde{R}$, so $T^{-1} = P\tilde{R}^{-1}D^{-1}Q^\dagger$. For `NVAR=1`:

1. `sA`$\leftarrow U_R^\dagger$ (`k_ct`); with `STABLOG` rows are scaled by e^{-L} where $L > 0$;
2. `la_ormqr('L','C')`: $Q^\dagger$ `sA`;
3. `k_rowdiv`: divide row i by D_i (Smith's complex division in the HIP kernel);
4. `la_trsm('L','U','N','N')` with $\tilde{R}$;
5. `k_iperm_rows`: undo P (`ZLAPMR` backward);
6. $G = U_L \cdot (\ldots)$ with `la_gemm`.

`NVAR=2` works on $T^\dagger$ with right-side operations. The cost is about $41N^3$ flops per `CGR` (derived). The result `st_Gn` is copied device-to-device into `G_d`; in the "lazy" mode (ticket 208-1) the host G is not written.

6.6 Precision monitoring

`Control_PrecisionG` compares the wrapped G with the fresh one. On the device (`dev_prec`, `k_prec`, `gpu_stab_mod.F90:743-754, 1524-1567`) one kernel computes

$$x_{\max} = \max_{ij} |G_{ij}^{\text{wrap}} - G_{ij}^{\text{new}}|, \quad x_{\text{sum}} = \sum_{ij} |G_{ij}^{\text{wrap}} - G_{ij}^{\text{new}}|, \tag{19}$$

and the number of NaN entries in each matrix (by a bit test, so that a NaN cannot hide in a comparison). The sum is in fixed order: one thread per column writes its column sum, then one thread adds the column sums. Four numbers go to the host, where `Control_PrecisionG_vals` (`control_mod.F90:463-526`) does what the CPU routine does: abort on NaN, abort if $x_{\max}$ exceeds the threshold (default 10, settable with `ALF_UNSTABLE_THRESHOLD`), otherwise accumulate mean and maximum.

The production threshold was lowered to 10^{-4} after a faulty GPU tile produced corrupted Green functions with deviations of 10 to 28, while healthy runs stay between 10^{-10} and 3×10^{-8} up to $L = 44$ (`DECISIONS.md`, item 46). The run then restarts from the last configuration within the job (at most 10 times).

6.7 The phase of the determinant

The phase of the fermion determinant is tracked as in `cgr1_mod.F90`. ALF computes it from two parts: the phase of $\det(U_R^\dagger U_L)$, obtained with an LU decomposition (`PHASE = CONJG(DET_C(RHS))`, `cgr1_mod.F90:303`), and the phase of $\det T$ from its QR factors.

Phase of $\det T$ from Householder scalars. LAPACK represents $Q = H_1 H_2 \cdots H_n$ with $H_i = I - \tau_i v_i v_i^\dagger$, $v_i(i) = 1$. By the matrix determinant lemma

$$\det H_i = 1 - \tau_i\, v_i^\dagger v_i. \tag{20}$$

H_i is unitary: $H_i^\dagger H_i = I - (\tau_i + \bar\tau_i) v_i v_i^\dagger + |\tau_i|^2 (v_i^\dagger v_i)\, v_i v_i^\dagger = I$, which requires $|\tau_i|^2\, v_i^\dagger v_i = 2\,\Re\tau_i$. Inserting,

$$\det H_i = 1 - \frac{2\tau_i \Re\tau_i}{|\tau_i|^2} = \frac{|\tau_i|^2 - 2\tau_i \Re\tau_i}{|\tau_i|^2}. \tag{21}$$

With $\tau_i = a + ib$ the numerator is $a^2 + b^2 - 2a^2 - 2iab = -(a+ib)^2 = -\tau_i^2$, so

$$\det H_i = -\frac{\tau_i^2}{|\tau_i|^2} = -\frac{\tau_i}{\bar\tau_i} \qquad (\tau_i \neq 0), \qquad \det H_i = 1 \quad (\tau_i = 0). \tag{22}$$

It has modulus one and does not depend on the normalisation of v_i; only the unitarity of H_i is used. The same formula is in upstream `cgr1_mod.F90:532-536`. The device kernel `k_cgr_phase` (`gpu_stab_mod.F90:1420-1452`) multiplies the phase by $R_{ii}/|R_{ii}|$ and by Eq. (22) for every i, and by the sign of the permutation. In code: `z = 1 - 2*(z/|z|)*(Re z/|z|)`, which is Eq. (22).

The LU-free phase of $\det(U_R^\dagger U_L)$ (2026-11-13). On the CPU, and still on NVIDIA and AMD, the device computes $\det(U_R^\dagger U_L)$ with `la_getrf` on the product and `k_det_phase` (product of the diagonal, sign flip for each row interchange). That is an extra N^3 factorisation per `CGR`, and on Intel oneMKL's `getrf` is slow (13 ms at $N = 1024$, `SPEEDUPS.md`). Since $\det(U_R^\dagger U_L) = \overline{\det U_R}\, \det U_L$, it suffices to track the phase of $\det U$ for every stored U. Every U is produced by a decompose, where $U = Q_{\rm ns}\,\mathrm{diag}(\phi, 1, \ldots, 1)$ and $Q_{\rm ns} = Q \cdot \frac{1}{2}(3I - Q^\dagger Q)$. The Newton–Schulz factor is Hermitian positive definite, so its determinant is real and positive and does not change the phase. Hence

$$\frac{\det U}{|\det U|} = \phi \prod_i \Big(-\frac{\tau_i}{\bar\tau_i}\Big) \Big/ \Big|\ldots\Big|, \tag{23}$$

which `k_phase` computes during the decompose at no extra cost (`gpu_stab_mod.F90:1024-1054`; line 1044 reads `if (a2 > 0) dq = dq*(1 - 2*z*(real(z)/a2))` with `a2`$= |z|^2$). The value is stored with the slot (`st_dw`, `st_dv`) and copied with it. In `CGR`, `k_det_phase_u` forms $\det = \overline{d_R}\, d_L$ and the phase $\bar d/|d|$, replacing the LU (lines 1076-1086). Reset slots have $\det = 1$. If a slot was loaded from the host (no tracked phase), the LU is used as fallback.

This path is enabled only with oneMKL (`#if defined(ALF_LA_ONEMKL)`, lines 471-477). The comment in the code: "enabled where it has been measured and verified: oneMKL (Max 1550, 2026-11-13: 4.45 → 4.04 s per sweep at L=24, phase equal to the LU phase to 6e-15). HIP: `hk_phase` does not track det(U). CUDA: not yet run." `SPEEDUPS.md` gives 5.8×10^{-15} and also 10.06→9.59 s with default switches. `ALF_GPU_CGR_LU=2` runs both routes and reports the largest difference.

One caveat follows from the derivation: Eq. (22) holds for the LAPACK representation. If the own device QR (Section 6.8) is used on Intel together with the LU-free phase, it must produce LAPACK-convention τ_i; the own kernels do follow `ZLARFG` conventions (`alf_la_sycl_devqr.hpp`), and the V7 measurements of 2026-11-13 used both together. **[TODO: a dedicated test of the LU-free phase with ALFSH_DEVQR=2 is not documented separately from the V7 run]**

Code and documents disagree. The header of `gpu_stab_mod.F90` (lines 46-49) says that the phase comes "from the determinant phases of the two U factors … la_getrf only as fallback". In the code only the oneMKL backend tracks $\det U$; CUDA and HIP always use `la_getrf`.

6.8 Vendor QR and own device QR

`la_geqrf` and `la_ormqr` are the most frequent large operations: at $L = 24$ about 217 calls per sweep (`alf_hip_qr.hip.cpp:3-4`). Their quality differs strongly between vendors:

- **cuSOLVER** is adequate: at $n = 1024$ `geqrf` 7.8 ms, `ormqr` 2.2 ms, `getrf` 4.1 ms, together about 8 % of a sweep; a CUDA port of the own QR was therefore not made (`SPEEDUPS.md`, section 3).
- **oneMKL** `geqrf` is hybrid: it factorises each panel on the host CPU and updates the trailing matrix on the device. At $L = 24$ this cost 2.4 s per sweep (`GPU_PLAN_V2.md`, section 2).
- **rocSOLVER** `zgeqrf`/`zunmqr` at $n = 576$ launch about 2300 tiny kernels each, 8.8 ms per call, about 0.12 TFlop/s on an MI210; QR was 93 % of all kernel dispatches and about 44 % of the sweep (`alf_hip_qr.hip.cpp:3-4`; `GPU_PLAN_V2.md`).

The own HIP QR (`alf_hip_qr.hip.cpp`, 347 lines) is a blocked right-looking Householder QR in LAPACK conventions (`ZLARFG` reflectors, R in the upper triangle, v below with implied unit diagonal, τ), so all consumers use its output unchanged.

- Panel kernel `k_qr_panel<NB,T,R>`: one work-group factorises the whole panel with the panel held in registers; row i lives in thread $i \bmod T$. $NB = 16$, $T = 256$ threads for $m \le 1024$; $NB = 8$, $T = 512$ for $m \le 2304$. Per column: the norm by a wave-level sum plus one exchange through local memory, `ZLARFG`, then the dot products with the remaining panel columns (three barriers per column). No T factor is formed.
- Apply kernel `k_apply_left<Q,NV>`: one 64-lane wave owns NV columns and applies the reflectors one after another (a dot product and a rank-one update per reflector). It does the trailing update of each panel in one launch and also implements `unmqr`. No compact-WY form and no GEMM.
- Right-side `unmqr` is computed as $(Q^\dagger C^\dagger)^\dagger$ through a transpose kernel.
- Limits: $m \le 2304$; otherwise hipSOLVER.

Measured at $n = 576$: `geqrf` 8.8→4.4 ms, `unmqr` 8.5→1.7 ms (`amd_mi210/README.md`). After this change the profile at $L = 24$ is: QR panel 35.9 %, rocBLAS GEMM 23.0 %, QR apply 18.8 %, `trsm` 8.6 % of 4.71 s kernel time with 118 303 dispatches per sweep (`amd_mi210/profile_L24_counters.txt`). The panel is the bottleneck: it runs on one compute unit of 104 while the others wait.

The own SYCL QR (Intel; `alf_la_sycl_devqr.hpp`, `alf_la_sycl_qr2.hpp`, loaded at run time from `libalfdevqr.so`) has two levels.

- `ALFSH_DEVQR=1`: the reference algorithm of `ZGEQRF`. The panel (`ZGEQR2` plus `ZLARFT`, compact-WY T factor) runs in one work-group with one 16-lane sub-group per panel column, the column held in registers. The owner of the next column computes `ZLARFG` from a norm published one step earlier (look-ahead). The trailing update is `ZLARFB` as three oneMKL GEMMs: $W = V^\dagger C$, $W_2 = T^\dagger W$, $C \leftarrow C - VW_2$. Panel width 32 by default (`ALFSH_QRNB`, at most 48: 64 needs 1024 work-items, which fail to launch on the Max 1550). Larger matrices than $M = 1536$ go to oneMKL.
- `ALFSH_DEVQR=2` ("phase 5", modelled on the HIP version): the panel without T (`panel2_t`), and an apply kernel (`apply2_t`) that applies a range of reflectors to independent columns, staging chunks of reflectors in shared local memory, double-buffered. One launch per panel for the trailing update. With `ALFSH_DEVUNMQR=1` the same kernel implements `unmqr`. Panel width 16 (32 for $M \le 256$).

Where the time goes, according to the comments: the level-1 panel kernel is bound by instruction issue on the single Xe core it runs on; without the column in registers, global memory latency (about 1 μs per dependent load) would dominate. Most of the 7 μs per column step of an earlier version went into the T recurrence ($O(i)$ dependent loads) and the dot products; dropping T (level 2) removed it. The apply kernels are bound by the bandwidth of local memory and by global latency (`alf_la_sycl_qr2.hpp:5-6, 238-241`).

Measured inside ALF at $L = 24$ on the Max 1550, per call: `geqrf` 5.48→3.91 ms and `unmqr` 2.4→1.37 ms with level 2; the sweep went from 6.53–6.59 s (V2) to 6.12–6.18 s (level 1, V4) and

4.95–4.97 s (level 2 plus device `unmqr`, V7) (`ISSUE_INTEL_VS_A30.md`, section 2). The own QR is not yet in production on Intel: `RUNTIME_SWITCHES.md` (line 124) lists it as "not yet"; the $L = 6$, `Mz=.T.` test with V7 gives 1.36×10^{-10} against a tolerance of 10^{-10} (`CONSOLIDATION.md`). On AMD the own QR is the default.

6.9 Transfers of the stabilisation

Per decompose and flavour: $8N$ bytes of norms D→H, $4N$ bytes of permutation H→D, 24 bytes of scale test. Per `CGR`: the same for the QR of T, 16 bytes of phase and 32 bytes of precision numbers. With `STABLOG`, `la_geqp3_host` adds $16N^2$ bytes each way per `CGR` (`gpu_stab_mod.F90:59-67`).

7 Local updates

The local updates are in `Prog/gpu_update_mod.F90` (633 lines: the loop over vertices, the ratio, the rank-one path) and `Prog/gpu_lowrank_mod.F90` (1723 lines: delayed and submatrix schemes, block mode, the asynchronous Intel path). This was the hardest part of the port and the part where the algorithm, not the kernel, decided the speed.

7.1 Why the rank-one update is slow on a GPU

Each proposal needs G_{pp} of the current G, which depends on all earlier acceptances. A device implementation of Eq. (9) must, per proposal, (i) copy G_{pp} to the host (or decide on the device), (ii) let the host decide, (iii) launch the update kernel. Each of these is a synchronous operation with a fixed cost (*latency*) of several microseconds, independent of N. Measured costs:

Table 8: Measured cost of single device operations.

Platform, operation	Source	μs
A30, empty OpenMP target region, synchronous	`BENCHMARKS.md`, ticket 102	12.4
A30, empty OpenMP target, `nowait`	same	11.0
A30, empty OpenACC kernel, `async`	same	2.7
A30, one proposal with host decision	same	15.2
A30, D→H→D round trip, $N \leq 400$	`ADR-001`	28–34
A30, rank-1 update $N = 144/400/1024$ (device)	`BENCHMARKS.md`	4.7 / 6.6 / 31.6
A30, rank-k ($k = 128$) per update, same N	same	0.21 / 0.37 / 1.25
Max 1550, empty synchronous target	`SYNC.md`	6.1
Max 1550, small kernel	same	9–12
Max 1550, ZGEMM $1024 \times 32 \times 32$ / $1024 \times 1024 \times 32$	same	17.7 / 55
Max 1550, D→H 16 B / 512 KB	same	2.8 / 350
Max 1550, SYCL submit, asynchronous	`p3/STATUS.md`	4.3
MI210, GCC (libgomp) target region with parallel loop	`alf_hip_kernels.hip.cpp:2`	≈150
MI210, HIP launch plus synchronisation	same	≈15

With $2NL_{\text{trot}}$ proposals per sweep and about 30 μs per proposal, a sweep at $L = 24$ ($N = 576$, $L_{\text{trot}} = 240$) would spend $2 \cdot 576 \cdot 240 \cdot 30$ μs ≈ 8 s in latency alone, before any arithmetic. The rank-k row of Table 8 shows the way out: applying $k = 128$ updates as one matrix product costs 25 to 40 times less per update. Two schemes that collect updates, the delayed and the submatrix update, exist as CPU code in `Lowrank_update_mod.F90` (taken into the branch on 2026-11-06 from a fork branch; the upstream master `521ddfca` has no such module). The port implements both on the device; the submatrix scheme in *block mode* is the production path.

7.2 Common part: the ratio and the decision

For all schemes the host does the same arithmetic as `Upgrade2` (`gpu_update_mod.F90:361-465`). Only single-site diagonal vertices are supported ($N_{\text{non-zero}} \le 1$). With $z_1 = g\,(\eta_{\text{new}} - \eta_{\text{old}})$, the vertex value E_1 and α:

$$\Delta = e^{z_1 E_1} - 1, \qquad d = 1 + \Delta\,(1 - G_{pp}), \tag{24}$$

$$R_f = d\; e^{z_1 \alpha}, \qquad R = \Big(\prod_f R_f\Big)^{N_{\text{SUN}}} \frac{\gamma_{\text{new}}}{\gamma_{\text{old}}}, \tag{25}$$

$$W = S_0\, T_0\, \left|\Re(\Phi R)/\Re\Phi\right|, \qquad \text{accept if } W > \texttt{ranf_wrap()}. \tag{26}$$

The random numbers are drawn on the host in the same order as on the CPU (the T_0 test draw, then the Metropolis draw). The only input that depends on the scheme is G_{pp}. On acceptance the update of G uses $z = \Delta/d$.

7.3 Standard (rank-one) update

G_{pp} is copied D→H (16 bytes). On acceptance two kernels run per flavour: `k_upd_prep` sets $x = G_{:,p}$ and $y = -G_{p,:}$ with $y_p = 1 - G_{pp}$, and `k_upd_geru` performs $G_{ij} \leftarrow G_{ij} - z\,x_i y_j$, which is Eq. (9) (`gpu_update_mod.F90:301-334`). The new field is written into `nsigma_d` by a 16-byte H→D. The complex scalar z is passed as two reals: nvfortran 26.9 dropped the imaginary part of a complex scalar argument inside a target region, which gave errors of 10^{-3} to 1 in G (`DISCREPANCIES.md`).

7.4 Delayed update

Representation. Per flavour $G = G_0 - XY^T$ with $G_0 =$ `G_d` and $X, Y \in \mathbb{C}^{N \times N_{\text{d}}}$ on the device; j columns are in use (`gpu_lowrank_mod.F90:39-45`).

Proposal. $G_{pp} = (G_0)_{pp} - \sum_{l \le j} X_{pl} Y_{pl}$, computed by a single-thread kernel; two reals go D→H.

Accept. One kernel computes the current column and row of G and appends them:

$$c = G_0(:,p) - X\,Y(p,:)^T = G(:,p), \qquad r = G_0(p,:) - X(p,:)\,Y^T = G(p,:), \tag{27}$$

$$X(:,j{+}1) = z\,c, \qquad Y(:,j{+}1) = e_p - r^T. \tag{28}$$

Then $G_0 - XY^T$ after the append equals $G - z\,G(:,p)\,(e_p^T - G(p,:))$, which is Eq. (9). By induction, after j acceptances $G_0 - XY^T$ equals j sequential rank-one updates; only the summation order differs.

Wrap. The vertex wrap $G \to LGR$ with diagonal L, R that differ from the identity only at site r is applied to G_0 (row r times z_a, column r times z_b) and to the factors, $X(r,:) \leftarrow z_a X(r,:)$, $Y(r,:) \leftarrow z_b Y(r,:)$. This is exact: $LGR = LG_0R - (LX)(RY)^T$.

Flush. $G_0 \leftarrow G_0 - X(:,1{:}j)\,Y(:,1{:}j)^T$, one `la_gemm` of size $N \times N \times j$, when $j = N_{\text{d}}$, before a vertex with $N > 1$ and at the end of every `WRAPGRUP`/`WRAPGRDO`. The hopping wraps, stabilisation and measurements always see a flushed G.

The delayed scheme removes the N^2 work per acceptance but still needs one round trip per proposal. A "fast" variant handles both flavours in one launch per site and defers the field write and the next wrap into the arguments of the next launch (`gpu_lowrank_mod.F90:699-733, 909-1055`).

7.5 Submatrix update

Representation. Per flavour, with the set $S = \{s_1, \ldots, s_k\}$ of distinct pending sites ($k \le N_\mathrm{d}$) and a small matrix $M \in \mathbb{C}^{k\times k}$ kept on the *host* (`gpu_lowrank_mod.F90:46-51`):

$$G = G_0 - G_0(:,S)\; M\; (E_S^T - G_0(S,:)), \tag{29}$$

where E_S is the $N \times k$ matrix of unit columns e_{s_m}. Between flushes G_0 is not changed by updates.

Proposal. Let $J = e_{\mathrm{slot}(p)} \in \mathbb{C}^k$ if $p \in S$ and $J = 0$ otherwise. From Eq. (29), since $(E_S^T - G_0(S,:))(:,p) = J - G_0(S,p)$,

$$G(:,p) = G_0(:,p) - G_0(:,S)\, w, \qquad w = M\,(J - G_0(S,p)), \qquad G_{pp} = (G_0)_{pp} - G_0(p,S)\, w. \tag{30}$$

This needs only $(G_0)_{pp}$, $G_0(S,p)$ and $G_0(p,S)$: $2k+1$ numbers. The code (`gpu_lr_gpp`) computes w and G_{pp} on the host in $O(k^2)$ and caches w and $G_0(p,S)$ for the accept.

Accept (host only). If $p \notin S$, S is padded with p and M with a zero row and column ($k' = k+1$, slot $i = k'$); otherwise $i = \mathrm{slot}(p)$ and $k' = k$. Define

$$a = e_i - \begin{pmatrix} w \\ 0 \end{pmatrix}, \qquad b^T = e_i^T + (G_0(p,S)\, M,\; 0), \qquad M \leftarrow M + z\, a\, b^T. \tag{31}$$

No device work at all (`gpu_lowrank_mod.F90:649-696`).

Equivalence with the rank-one update. With the padded set S' and $C = G_0(:,S')$, $Q = E_{S'}^T - G_0(S',:)$: since $p \in S'$, $G_0(:,p) = Ce_i$, hence $G(:,p) = C(e_i - [w;0]) = Ca$. Further $e_p^T - G_0(p,:)$ is row i of Q, and $G_0(p,S)MQ_S = [G_0(p,S)M, 0]\, Q$, hence $e_p^T - G(p,:) = b^T Q$. The Sherman–Morrison step becomes

$$G' = G - z\, G(:,p)(e_p^T - G(p,:)) = G_0 - C\,(M + z\, ab^T)\, Q, \tag{32}$$

which is the update of M above. The formula covers new, repeated and overlapping sites in one expression, as on the CPU (`Lowrank_update_mod.F90:1187-1198`).

Wrap. $G \to LGR$ with L, R diagonal and different from I only at site r: $G_0 \leftarrow LG_0R$ on the device, and if $r \in S$, row slot(r) of M times z_a and column slot(r) times z_b on the host. Writing $G_0' = LG_0R$ one finds $LG_0(:,S) = G_0'(:,S)R_S^{-1}$ and $(E_S^T - G_0(S,:))R = R_S E_S^T - L_S^{-1}G_0'(S,:)$. The representation is preserved with $M' = L_S M R_S$ exactly when $R_S = L_S^{-1}$, i.e. $z_a z_b = 1$. For the vertices supported this holds: $z_a = E(s)$ and $z_b = E(-s)$ with $\eta(-s) = -\eta(s)$. The code does not assert it.

Flush. With $C = G_0(:,S)$ and $Q^T = E_S - G_0(S,:)^T$ gathered by one kernel into `X_d` and scratch,

$$Y = Q^T M^T\; (N \times k \times k), \qquad G_0 \leftarrow G_0 - C\, Y^T\; (N \times N \times k), \tag{33}$$

two `la_gemm` calls (`gpu_lowrank_mod.F90:480-497`). $C\,Y^T = CMQ$ is the correction of Eq. (29).

N_d. If `N_delay`≥ 1 it is used as given; otherwise the static rule of the CPU (`Lowrank_static_N_delay`) applies, and on the GPU the submatrix scheme additionally uses at least $\min(32, n_\mathrm{nz})$: "larger blocks amortise the device round trips" (`gpu_lowrank_mod.F90:253`). `N_delay=-1` (automatic tuning on the CPU) is treated like 0 on the GPU.

7.6 Block mode

The submatrix scheme as described still needs one gather and one D→H per proposal. Block mode removes them (`gpu_lowrank_mod.F90:1253-1260, 1431-1720`). It is active when the scheme is submatrix and the "fast" conditions hold: $N_{\mathrm{FL,eff}} \le 2$, every vertex a single-site operator, and the sites of different vertices distinct (`fast_check`).

Idea. Between flushes G_0 changes only by the diagonal wrap scalings, and a proposal at p needs only $(G_0)_{pp}$, $G_0(p,S)$, $G_0(S,p)$ with $S \subset B$, the sites of the block. So the host can copy the principal block $G_0(B,B)$ once, apply each wrap to its copy, and decide all proposals of the block without the device.

A block is $b = \min(N_\mathrm{d}, \text{remaining vertices of the slice})$ consecutive vertices in sweep order (ascending in `WRAPGRUP`, descending in `WRAPGRDO`); blocks do not cross slices. Since the sites are distinct, $k \le b \le N_\mathrm{d}$, so no capacity flush can happen inside a block.

Sequence of one block.

1. `blk_begin`: the host builds the list of block sites; H→D of the list; kernel `k_blk_gather` copies $G_0(B,B)$ of both flavours into `gbd`; D→H of `gbd` ($32N_\mathrm{d}^2$ bytes).
2. For each vertex, on the host only: wrap of the host block (row a times z_a, column a times z_b; the factors are recorded) and of M; proposal from the host block by the formulas above; accept by the host update of M. On the way up the wrap precedes the proposal; on the way down it follows the update with the final field.
3. `blk_end`: the host packs the recorded factors and M into `cbuf`; H→D of `cbuf` and of S; kernel `k_blk_scale` applies all row factors and then all column factors of the block sites to G_0 (both flavours, in the order of the two-kernel version); per flavour the flush (gather plus two GEMMs); H→D of the new fields `nsigma_d(vmin:vmax,t)`.

A flush with $k = 1$ is padded to $k = 2$ with a duplicate site and a zero row and column of M (`gpu_lowrank_mod.F90:1528-1532`); the extra site contributes nothing to CMQ because its row and column of M are zero, so the result is the same as a rank-one flush up to rounding. Neither the code nor the commit that introduced it (`e70a3061`) gives a reason **[TODO: reason for the k=1 to k=2 padding in blk_end; possibly to avoid a library special case for $k = 1$, not documented]**.

At $L = 24$, $N_\mathrm{d} = 32$ one sweep has 8 640 blocks (18 per slice × 480 slices) with 8 640 gathers, 17 280 flush gathers, 8 640 scale kernels, 34 560 GEMMs and 43 200 copies (`p7/NOTES.md`, line 94). The flush rank is small: k of 10 to 14 on average (`p7/NOTES.md`, line 36).

Effect. On the A30 at $N = 400$ the per-proposal path cost 117 μs per proposal (5.0 of 7.9 s); block mode brought this part to 0.53 s, ten times less (`DELAYED_SUBMATRIX.md`, line 52). The submatrix scheme with block mode is 2.4 to 5 times faster than the standard update and 1.4 to 2.1 times faster than the delayed scheme at the same N_d ($L = 16$ to 32, A30, `TUNING_DEFECT.md`).

On nvfortran 26.9 the routine `blk_end` is compiled with `!pgi$r opt=1`: at `-O2`/`-O3` the compiler produced a wrong G after the first stabilisation (`gpu_lowrank_mod.F90:1496-1499`).

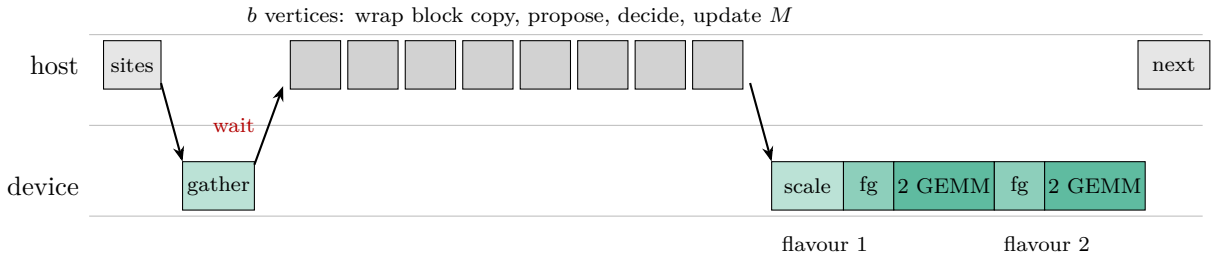


Figure 2: One block of the submatrix update in block mode (schematic, not to scale). Host work is grey, device work green; "fg" is the flush gather. The only unavoidable wait is the copy of $G_0(B,B)$ to the host at the beginning of the block, because the next gather depends on the previous flush. In the synchronous path every arrow is also a wait.

7.7 Synchronisation points per block

In the synchronous OpenMP path every `target` region and every `target update` returns only when it is complete. In addition `alf_la` calls run asynchronously on NVIDIA (cuBLAS returns before the GEMM is done), so two explicit `la_sync` calls are needed. The sequence per block (`gpu_lowrank_mod.F90:1443-1592`):

1. `la_sync` in `blk_begin`: the previous flush (asynchronous library calls) must be complete before new uploads overwrite buffers it reads.
2. `target update to(bsd)`: the block sites.
3. `k_blk_gather`.
4. `target update from(gbd)`: the host needs $G_0(B, B)$ before it can decide anything. This is the one wait that cannot be removed.
5. `la_sync` in `blk_end`: "the previous block's flush (cuBLAS) is asynchronous: it must be done before cbuf / bS_d are overwritten".
6. `target update to(cbuf, bS_d)`.
7. `k_blk_scale`.
8. Per flavour: `k_sub_flush_gather` and two `la_gemm`. On Intel the dispatched GEMM includes an implicit wait (`SYNC.md`).
9. `target update to(nsigma_d(vmin:vmax,t))`.

On the A30 a profile of a production layout showed 72 % of host time in `cuStreamSynchronize`, with about 4 300 blocks and 44 000 synchronisations per sweep (`PP_GPU.md`, $L = 24$, 9 ranks). In the single-chain profile at $L = 32$, 44 % of the kernel time was in 32 338 small ZGEMM flushes of 52 μs each (`TUNING_DEFECT.md`).

7.8 Launch-latency budget of a block

The Intel team measured where the 167 μs of wall time of one block go at $L = 24$ with V7 (`p7/NOTES.md`, line 97): submission of the asynchronous operations 91 μs, the D→H copy 22 μs (30 at $L = 32$), the hopping 32 μs, host arithmetic about 20 μs; the device executes the block in about 65 μs. The update phase is therefore "host-submission bound": the cost of telling the device what to do exceeds the work it does. The per-operation submission costs were 4.4 μs (gather), 4.5 (flush gather), 4.4 (scale), 9.7 (GEMM) and 9.0 (copy), 0.88 s per sweep in total, 61 % of the update phase (`p7/NOTES.md`, lines 88-97).

Before the staging buffer setting the D→H of the 32 KB block cost 380 μs on Intel; with `LIBOMPTARGET_LEVEL_ZERO_STAGING_BUFFER_SIZE=1048576` it costs 22 μs, and the update phase went from 4.3 to 1.9 s per sweep (`p3/STATUS.md`; `ISSUE_INTEL_VS_A30.md`, line 43). "One environment variable gained more than two code phases" (`GPU_PLAN_V2.md`, section 2).

7.9 The asynchronous block path on Intel

On NVIDIA, cuBLAS calls are asynchronous by default. On Intel, ifx executes OpenMP `nowait` target tasks undeferred (97 to 103 % of the synchronous cost, `p3/STATUS.md`), so OpenMP cannot overlap host and device. The asynchronous block path (`ALFSH_ASYNCBLK=1`, `gpu_lowrank_mod.F90:1328-1347, 1605-1617`; `alf_la_sycl_blk.cpp`) bypasses OpenMP for the block operations:

- It obtains the SYCL queue that the OpenMP runtime uses (`alfsh_get_queue`, through the interop object of `alf_la`) and refuses it if it is not *in-order*. "The queue must be in-order: that, and nothing else, orders the calls" (`alf_la_sycl_blk.cpp:4`).
- The gather, scale and flush-gather kernels are SYCL ports of the OpenMP kernels with the same expressions and order; the GEMM is oneMKL's `column_major::gemm` with the same shapes and scalars; copies are `queue.memcpy`. All are submitted without waiting. The gathered

block lands in host USM memory (memory that the device can write directly).
- The only wait per block is `ab_drain` in `blk_begin` after the gather: "the one unavoidable wait of a block: the host needs G0(B,B)". `blk_end` leaves the block pending; the next `blk_begin`, the flush at the end of the slice, or any other device access drains it.
- The SYCL code is built as a shared library (`libalfsb.so`) because SYCL kernels in an object linked by ifx are not found at run time ("No kernel named ...").

Results are bitwise identical to the synchronous path. Planted tests: a missing wait gives an "Unstable matrix" abort; a missing drain at the end of the slice changes nothing, because the in-order queue orders the OpenMP kernels too (`p3/STATUS.md`). Measured effect: $L = 24$ 7.05–7.08→6.53–6.59 s per sweep, $L = 32$ 23.2→22.0–22.4 s, per card 0.568→0.655–0.688 sweeps/s (`ISSUE_INTEL_VS_A30.md`, section 2).

Code and documents disagree. The release status of `ALFSH_ASYNCBLK` differs between notes: `RUNTIME_SWITCHES.md` (line 123) lists it as production, `SPEEDUPS.md` (line 21) and `NOT_IMPLEMENTED.md` (line 118) as not yet released. The measured gain is +3 % per node in `p3/STATUS.md` and +15 to 20 % per card in `ISSUE_INTEL_VS_A30.md`; the jobs differ.

7.10 What remains: the flush

In production layouts the flush GEMMs are 47 % of the kernel time (A30, 9 ranks, `PP_GPU.md`). They are rank-k updates with $k \approx 10$ to 20, bound by memory bandwidth (N^2 elements read and written per flush for $2Nk$ new information). The ranked candidate is a two-level submatrix update: one flush per slice of rank about 190 instead of one per block (`SPEEDUPS.md`, section 4). It is estimated at weeks of work and not started.

Code and documents disagree. The header of `gpu_lowrank_mod.F90` (lines 34, 52-53) calls the module a "test branch" where "numerical agreement is not the goal". It is the production path, and `DELAYED_SUBMATRIX.md` reports bin-by-bin agreement. The header also describes only the per-proposal gather; block mode is documented only at line 1253.

8 Observables

Measurements in ALF are model code: each Hamiltonian's `Obser` and `ObserT` read G and accumulate into observable objects. A general port would need a device version of every model's measurement routine. The port does this for the predefined Hubbard model only (`Prog/gpu_obs_mod.F90`, 785 lines; time-displaced part in `gpu_tau_mod.F90`). For other models the GPU path downloads G and the host measures.

8.1 Hook-up and conditions

`gpu_obs_setup` is called at the end of `Alloc_obs` of `Hamiltonian_Hubbard_smod.F90` (lines 788-789) with the lattice, the hopping matrix and the parameters. `Obser` begins with

```
if (gpu_obs_active()) then; call gpu_obs_measure(...); return
```

(lines 831-837). `gpu_obs_active` requires: setup done, the model checks passed, `ALF_GPU_NOOBS` not set, the GPU path and the device update active. The model checks (`gpu_obs_mod.F90:92-184`) are: no projector; $N_{\mathrm{FL}} \in \{1, 2\}$; the expected number of equal-time observables (5 for $N_{\mathrm{FL}} = 2$, else 3); $N = N_{\mathrm{cell}} N_{\mathrm{orb}}$; the displacement table `Latt%imj` a bijection per site; with the pinning field ($h_0 \neq 0$) $N_{\mathrm{FL}} = 2$ and two local observables. If a check fails the run measures on the host; this is a silent fallback, not a refusal.

In `main.F90` (lines 1033, 1109) the download of G before the measurement is skipped when the device measures, and `Hop_mod_Symm` is skipped when the device applies `Symm`.

8.2 What is measured

Let $\bar{G}_{ij} = \delta_{ij} - G_{ji}$ (`GRC` in ALF) and $w = Z_P Z_S$ the phase and sign weight, passed to the kernels as two reals.

Scalars (`gpu_obs_mod.F90:300-341`):

$$Z_{\mathrm{kin}} = N_{\mathrm{SUN}} \sum_k h_k \, \bar{G}_{i_k j_k f_k}, \tag{34}$$

$$Z_{\mathrm{pot}} = \sum_{I,o} U_o \, (1 - G_{ii,\uparrow})(1 - G_{ii,\downarrow}), \qquad Z_\rho = N_{\mathrm{SUN}} \sum_{i,f} (1 - G_{ii,f}), \tag{35}$$

and the total energy $Z_{\mathrm{kin}} + Z_{\mathrm{pot}}$. The kinetic energy is a sparse linear functional of G, built once on the host as a list of (h_k, i_k, j_k, f_k) from the hopping matrix (a copy of `Predefined_Hoppings_Compute_Kin`, without the pinning term).

Correlation functions (`gpu_obs_mod.F90:404-525`). For $N_{\mathrm{FL}} = 2$ the kinds are Green, SpinZ, SpinXY, SpinT and Den; for $N_{\mathrm{FL}} = 1$ Green, SpinZ (SU(N)) and Den. For each pair of sites (i, j) with displacement $d = i - j$ and orbitals (o, o') the kernel accumulates into `acc`$(d, o, o', \mathrm{kind})$ the expressions of `Predefined_Obs_eq_*_measure`, for example

$$\text{Green:} \quad N_{\mathrm{SUN}} \sum_f \bar{G}_{ij,f}, \qquad \text{SpinSUN:} \quad N_{\mathrm{SUN}} \, \bar{G}_{ij} G_{ij}, \tag{36}$$

$$\text{Den:} \quad z_i z_j + N_{\mathrm{SUN}} \sum_f \bar{G}_{ij,f} G_{ij,f}, \qquad z_i = N_{\mathrm{SUN}} \sum_f (1 - G_{ii,f}), \tag{37}$$

and for $N_{\mathrm{SUN}} = 1$ the zz, xy and $T = (2xy + zz)/3$ spin correlations in one kernel.

The `acc0` terms. ALF stores with each correlation function the one-site expectation per orbital, `Obs_Latt0`, which the analysis subtracts to obtain connected correlations. The device keeps it in `acc0`$(N_{\mathrm{orb}}, n_{\mathrm{obs}})$: for Den, $N_{\mathrm{SUN}} \sum_I \sum_f (1 - G_{ii,f})\, w$; for SpinZ, $\sum_I [(1 - G_{ii,2}) - (1 - G_{ii,1})]\, w$, and the same divided by $\sqrt{3}$ for SpinT; Green and SpinXY have none (`gpu_obs_mod.F90:471-523`; host: `Predefined_Obs_mod.F90:190-191, 322`). The `acc0` sum used to run serially on one GPU thread, 760 μs per call and 7 % of the kernel time at $L = 24$; it is now one team per orbital with a parallel reduction (comment at lines 471-472). On NVIDIA this gave no measurable gain (88.9 s against 88.9 s, `PP_GPU.md`); on Intel it gave 4.92–4.98→4.42–4.48 s per sweep at $L = 24$ (`SPEEDUPS.md`, section 1). Results changed by at most 3.3×10^{-16} (`CONSOLIDATION.md`).

Point-defect observables (`gpu_obs_mod.F90:361-402`). Per site and orbital $d = (G_{ii,2} - G_{ii,1})\, w$ is accumulated into `lacc1` (`SpinZ_local`) and into `lacc2`$(\cdot, \tau, \cdot)$ (`SpinZ_localtau`) with index $\tau + 1 - T_{\mathrm{htrot}}$, weight 2 at the end points; the counter is incremented at $\tau = L_{\mathrm{trot}}/2$. This mirrors `Hamiltonian_Hubbard_smod.F90:901-922`.

8.3 How the reductions are organised

A reduction (a sum of many numbers into one) is the operation where a GPU and a CPU differ most. A naive parallel sum with *atomic* additions is non-deterministic: the order of additions, and so the rounding, changes from run to run. The port avoids atomics everywhere:

- **Scalars** (OpenMP path): one `target teams distribute parallel do reduction(+:...)` kernel per scalar.
- **Correlations** (OpenMP path): one *team* (a group of threads that can synchronise) per output element $m = (d, o, o')$, via `target teams distribute` over $N_{\mathrm{cell}} N_{\mathrm{orb}}^2$ elements; inside the team a `parallel do reduction(+:...)` over the N sites; then one thread adds the result into `acc`. Each output element belongs to exactly one team, so there are no conflicts.

- **HIP path** (AMD): `hk_scalars` is one block of 128 threads with strided partial sums and a tree reduction in shared memory; six doubles come back in one copy. `hk_corr` launches one block per output element plus N_{orb} blocks for the `acc0` sums, each with a tree reduction (`alf_hip_kernels.hip.cpp:157-262`).

The work per measurement is $O(N^2)$ per kind (all pairs i, j), the same as on the CPU.

Transfers. G is read from `G_d` directly. The reduced scalars come back per measurement. The accumulators `acc`, `acc0`, `lacc1`, `lacc2`, `tacc`, `tacc0` are downloaded once per bin by `gpu_obs_flush` (called from `main.F90:1337` before `Pr_obs`), zeroed and uploaded. Counters, the scalar accumulation into `Obs_scal`, normalisation, binning, the MPI reduction and I/O stay on the host.

Symm. With the symmetric Trotter decomposition, G is saved, transformed in place by $A(A_{td}GB_{td})B$ (two to four GEMMs) and restored after the measurement (`gpu_obs_mod.F90:268-284, 354-358`).

Skipping. `ALF_GPU_OBS_SKIP` takes a list of kinds 1 to 4 (Green, SpinSUN, Den, SpinMz) that are not measured and stay zero. Skipping Green and Den saved 7 % under MPS (`PP_GPU.md`). It is meant for production runs that do not need those observables.

Code and documents disagree. The header of `gpu_obs_mod.F90` says the supported case needs "$|h_0| <$ Eps_small; no Symm" (line 12), that the sum over I is "a sequential loop per output element" (line 20), and that the scalars are "one D→H of three complex numbers per slice" (line 16). The code supports the defect and `Symm`, sums with a parallel reduction inside each team, and on the OpenMP path returns three separate reduction results.

8.4 The time-displaced pass

With `Ltau=1`, `tau_m_mod.F90:82-83` calls `gpu_tau_pass` when the device τ pass is on. Conditions (`gpu_tau_mod.F90:234-283`): device stabilisation on, `Ltau=1`, no projector, no weight reconstruction. Setup reserves all library work space and runs one $2N$ QR so that nothing is allocated during the sweep.

Algorithm. `tau_begin` sets on the device $G(0,0) = G(\tau,0) = G(\tau,\tau) = G$ and $G(0,\tau) = -(1-G)$, keeps a copy of G as precision reference, and resets the right UDV slot. For each slice (`tau_slice`, lines 448-467):

$$G(\tau{+}1,0) = B_{\tau+1}\, G(\tau,0) \qquad (\texttt{gpu_mmthr}, \texttt{gpu_Vslice_R}), \tag{38}$$

$$G(0,\tau{+}1) = G(0,\tau)\, B_{\tau+1}^{-1} \qquad (\texttt{gpu_mmthl_m1}, \texttt{gpu_Vslice_L}), \tag{39}$$

$$G(\tau{+}1,\tau{+}1) = B_{\tau+1}\, G(\tau,\tau)\, B_{\tau+1}^{-1}. \tag{40}$$

At every stabilisation point the right state is propagated and decomposed on the device, the left state is read in place from the stack slot (no copy), and `dev_cgr22` recomputes the four Green functions from the $2N \times 2N$ block matrix of `CGR2_2`. The steps of `dev_cgr22` (lines 574-678): the branch $D_1(1) > D_2(1)$ from two scalars D→H; V_1^{-1} by LU (`getrf` plus `getrs` on the identity, where the CPU uses `ZGETRI`); the scale splitting; the $2N \times 2N$ matrix; QR with pre-pivoting (or host `ZGEQP3` with `STABLOG`); triangular solve and scaling; QX through an explicit Q with a Newton–Schulz step and a GEMM. With cuSOLVER's `ZUNMQR` applied directly the precision of the τ pass was 17 % worse on average and up to 2 times worse at the maximum than on the CPU; with the explicit Q and the step it was 6.75×10^{-12} against 7.36×10^{-12} on the CPU (header lines 56-59). The four precision checks of `Control_Precision_tau` run as device reductions, four numbers each.

Observables. If `gpu_obs_tau_active()`, every slice calls `gpu_obs_tau_measure` with the four device matrices. The kernel `tcorr` (`gpu_obs_mod.F90:648-783`) accumulates the same kinds as the equal-time measurement, resolved in τ, into `tacc`$(N, N_\tau, N_{\mathrm{orb}}, N_{\mathrm{orb}}, n_{\mathrm{obs}})$, e.g. SpinSUN $-N_{\mathrm{SUN}}\, G(0,\tau)_{ji}\, G(\tau,0)_{ij}$. This removes a D→H of $3N^2$ complex numbers per slice and flavour. The `tacc0` loops are still serial over N per orbital; under HIP `tcorr` stays an OpenMP kernel with a serial inner sum, unlike the equal-time HIP kernels.

What is refused. The device τ pass is off with `Ltau`$\neq 1$, the projector, weight reconstruction or without device stabilisation; then the CPU pass runs on downloaded data. The device τ observables are off with the point defect; in practice this case cannot occur because `Ham_Set` sets $h_0 = 0$ when `Ltau=1` (`Hamiltonian_Hubbard_smod.F90:247-259`). Debug and fault variables of the τ pass are refused in production runs.

Code and documents disagree. The header of `gpu_tau_mod.F90` (line 46) and the info line printed at setup (line 275) still say "observables on the host, interim"; since ticket 208-3 they are measured on the device. `TUNING_DEFECT.md` also says "host ObserT". The device τ observables apply only the static `Symm` matrices, while the host path calls `Hop_mod_Symm` with the slice arguments; this matters only with time-dependent hopping, which is switched off for `Ltau=1`. `gpu_obs_tau_active` does not test `gpu_upd_enabled`, while `gpu_obs_active` does: with `ALF_GPU_NOUPDATE=1` the equal-time observables move to the host and the τ observables stay on the device. `main.F90:1301` downloads G before every τ pass although the device pass does not need it.

Evidence. `Ltau=1` has been run on NVIDIA; it has not been run on the Max 1550 or the MI210 according to `FEATURES.md` (lines 61, 97).

9 MPI, parallel parameters and sharing a GPU

9.1 What MPI does in a GPU run

ALF's MPI parallelism is over independent Markov chains: each rank runs its own sweep, and only the observables are reduced at the end of a bin. The GPU port keeps this. Every rank builds its own device mirror and its own device G and stacks, draws its own random numbers on the host, and talks to the other ranks only through the host-side observable reduction (`gpu_state_mod.F90:516-519`). No GPU-aware MPI is needed.

Rank-to-device binding. The code sets the OpenMP default device to

```
call omp_set_default_device(mod(env_rank, omp_get_num_devices()))
```

(`gpu_state_mod.F90:550-552`), where `env_rank` is the rank in `MPI_COMM_WORLD`. In practice the job scripts restrict each rank to one device before the program starts, so the rule always yields device 0:

- NVIDIA (Helma): a wrapper sets `CUDA_VISIBLE_DEVICES` = r/ranks per GPU from the Open MPI rank r, and with MPS the pipe and log directory of that GPU's MPS daemon (`docs/gpu/scripts/helma/rank_wrap.sh`). The device used by the OpenACC stream query (Section 3) must be the OpenMP default device, which this binding guarantees.
- Intel (SuperMUC-NG Phase 2): `ZE_FLAT_DEVICE_HIERARCHY=FLAT` exposes the 8 tiles of a node's four Max 1550 cards as 8 devices, so rank r uses tile r mod 8; with fewer ranks `ZE_AFFINITY_MASK` binds them (`intel_pvc_supermuc_ng2.md:26-27`).
- AMD: no MPI run is documented (`FEATURES.md`, line 47).

Code and documents disagree. The binding uses the global rank, not the node-local rank. On several nodes the mapping is right only if the ranks are placed in blocks and the number of ranks per node is a multiple of the number of devices. This is not documented; the job scripts avoid the issue by setting the visible devices per rank.

Correctness. One MPI rank reproduces the non-MPI run to 10^{-12} (energy 7×10^{-12}, `SpinZ_localtau` 1.4×10^{-12}, `SpinZ_local` 7×10^{-15}), and acceptance (0.3375) and precision (10^{-14}) do not depend on the number of ranks (`MPI_GPU.md`).

9.2 Parallel parameters and tempering

With `TEMPERING` and `PARALLEL_PARAMS` ALF runs groups of ranks with different parameter sets. Real parallel tempering (exchange of configurations between parameter sets) is refused on the GPU path. Parameter-parallel runs without exchange are allowed: the groups are independent, and `main.F90` skips `Exchange_Step` (`PP_GPU.md`). Build: `configure.sh PGI PP GPU=nvidia HDF5` with `PP = -DMPI -DTEMPERING -DPARALLEL_PARAMS`.

9.3 Several ranks on one GPU

One chain does not fill a GPU: on the A30 one chain kept the GPU busy 58 % of the time, two chains 82 % (`MPI_GPU.md`, `nvidia-smi dmon`); on the H100 at $L = 32$ one chain used 55 % of the streaming multiprocessors (`helma_h100.md`, section 4.2). The reason is the structure of the work: most kernels are small (block gathers, rank-k flushes, single-vertex wraps) and the host spends time between them. Running several chains per GPU fills the gaps. How well this works depends on how the GPU shares itself between processes.

NVIDIA: MPS. Without help, processes on one NVIDIA GPU are *time-sliced*: the GPU switches between their contexts, and kernels of different processes never run at the same time. The *Multi-Process Service* (MPS) merges the processes into one context, so their kernels run concurrently. `CUDA_MPS_ACTIVE_THREAD_PERCENTAGE` limits the share of the GPU each client can use. Measured:

- Without MPS, two processes on the A30 ran 13 to 15 times slower each (`CLAUDE.md`); on the H100 two ranks were 6.5 times slower per chain at $L = 24$ (29.5 against 4.54 s per sweep) and 2.6 times at $L = 32$; four or eight ranks at $L = 32$ did not finish six sweeps in 420 s (`helma_h100.md`, section 4).
- With MPS the throughput rises with the number of ranks. Table 9 gives the A30 at $L = 16$.
- A 25 % thread share was 12 % faster than no limit with 9 ranks at $L = 24$ (86.9 against 98.3 s for 10 sweeps, `PP_GPU.md`); 25 % and 50 % agree within 2 % on the H100.

Table 9: A30, $L = 16$, submatrix $N_\mathrm{d} = 32$, 30 sweeps, with MPS (`MPI_GPU.md`).

Ranks	MPS share	Time per chain (s)	vs 1 rank	Sweeps/s per GPU
1	100 %	38.8	–	0.73
2	50 %	45.3	+17 %	1.25
4	25 %	54.6	+41 %	2.09
8	13 %	81.4	+110 %	2.83
16	default	145.6	+275 %	3.18
16	25 %	–	–	4.68

On the H100 at $L = 24$ one rank gives 0.285 sweeps per second, eight ranks with MPS at 25 % give 1.176, 4.1 times more, while each chain becomes 1.9 times slower (6.80 against 3.51 s per sweep). At $L = 32$: 0.102 with one rank, 0.232 with eight and 0.250 with sixteen (`helma_h100.md`, sections 4.1-4.2). MPS does not change the results: two MPS clients produced output bit-identical to a solo run (`CLAUDE.md`).

Intel: CCS. The Max 1550 has no MPS. Ranks sharing a tile are time-sliced by default. The compute command streamer setting `ZEX_NUMBER_OF_CCS=0:4,...,7:4` splits each tile into four hardware queues, so four processes can run concurrently on one tile. With the V7 switches at

$L = 24$ the throughput per card is 0.365, 0.458, 0.879 and 0.261 sweeps per second for 1, 2, 4 and 8 walkers per tile (`ISSUE_INTEL_VS_A30.md`, section 7). Four walkers per tile (32 ranks per node) is the optimum.

Code and documents disagree. `intel_pvc_supermuc_ng2.md` (line 45, 2026-11-10) reports that CCS 4 gave "no gain" and recommends one rank per tile (line 47); `RUNTIME_SWITCHES.md` (line 173) and the later session notes report CCS 4 with 32 ranks per node as the throughput optimum. The later measurement is the one used in production.

AMD. Four concurrent processes on the MI210 without any sharing service gave 1.95 times the throughput of one (`SPEEDUPS.md`, section 3); compute-unit masking (`HSA_CU_MASK`) gave nothing.

9.4 The production layout

The production layout is a measured per-platform setting (`GPU_PLAN_V2.md`, section 2): on the H100, eight ranks per GPU with MPS at 25 %, $N_\mathrm{d} = 64$ at $L = 32$ (`helma_h100.md`, section 7); on the Max 1550, four ranks per tile with CCS 4; on the A30 the owner decided on one rank per GPU (2026-11-08, `MPI_GPU.md`). The choice of N_d interacts with the layout: with eight ranks on the H100 at $L = 32$, $N_\mathrm{d} = 32$ gave 66.5 s per sweep per chain and $N_\mathrm{d} = 64$ gave 34.5 s (`helma_h100.md`, section 4.2).

10 The three backends in detail

The physics code is the same for all three vendors. What differs is the compiler, the library behind `alf_la`, and, on Intel and AMD, a layer of native kernels. Table 10 gives the overview; the subsections give the problems met and how they were solved.

Table 10: Backends as built and run.

	NVIDIA	Intel	AMD
Macro	`ALF_LA_CUDA`	`ALF_LA_ONEMKL`	`ALF_LA_HIP`
Hardware run	A30 (nividia), H100 (Helma)	Max 1550 (SuperMUC-NG Phase 2); Arc B580 for tests	MI210
Compiler	nvfortran 26.9 (A30), nvhpc 25.7 (H100)	ifx 2025.1 (production), 2025.3	gfortran 14 with amdgcn offload
Offload flags	`-mp=gpu -gpu=cc80|cc90`	`-fiopenmp -fopenmp-targets=spir64`	`-foffload=amdgcn-amdhsa`
BLAS/LAPACK	cuBLAS, cuSOLVER	oneMKL (OpenMP dispatch + SYCL shim)	hipBLAS, hipSOLVER, own HIP QR
Native kernels	none	SYCL: `libalfsb.so`, `libalfdevqr.so`	HIP: `alf_hip_kernels`, `alf_hip_qr`
Stream handling	`acc_async_sync` stream	interop in-order SYCL queue	default stream, deferred sync

10.1 NVIDIA

Toolchain. On the development machine nividia (A30, 24 GB) nvfortran 26.9; on Helma (4×H100 per node, 95.8 GB) nvhpc 25.7 with Open MPI 5.0.8 and `-gpu=cc90,cuda12.9` (`helma_h100.md`, section 2). cuBLAS and cuSOLVER are linked with `-cudalib=cublas,cusolver`.

Problems and solutions.

- **Complex scalar arguments in target regions.** nvfortran dropped the imaginary part

of a complex scalar `intent(in)` dummy inside a target region; G was wrong by 10^{-3} to 1. All complex scalars passed to kernels are now two reals (`DISCREPANCIES.md`; `gpu_update_mod.F90:316-317`, `gpu_obs_mod.F90:25`).

- **Stream ordering.** A created CUDA stream did not order cuBLAS calls against OpenMP target regions. The fix is the stream `acc_get_cuda_stream(acc_async_sync)`; the unit test `t_order` detects a wrong choice (`alf_la_mod.F90:48-61`). nvfortran does not implement `!$omp interop` (warning W-0287), hence the detour through the OpenACC runtime.
- **Alignment.** cuBLAS crashed when complex `alpha`/`beta` were only 8-byte aligned; they are now held in a 16-byte aligned module array (ticket 101).
- **Miscompilation of `blk_end`.** At `-O2`/`-O3` nvfortran 26.9 produced a wrong G after the first stabilisation; the routine is pinned to `-O1` (`!pgi$r opt=1`).
- **Stale `target update from`.** At `-fast` a `target update from` of a module allocatable returned stale data, and a fourth $N \times N \times N_{\mathrm{FL}}$ array in `gpu_state_mod` crashed the compiler (heap corruption in `fort1`) when compiling `gpu_stab_mod`. Workaround: helper routines `upd_*` and moving `GOO_d` to `gpu_tau_mod` (`gpu_tau_mod.F90:103, 141-144, 515`).
- **`present` map modifier** is rejected by nvfortran 26.9; `map(alloc:)` is used.
- **nvhpc 24.11** on Helma needs CUDA 12.6 (CUDA 11.8 lacks `cusolverDnSetDeterministicMode`) and then aborts at $L = 8$ with "la_qrpp: NaN or Inf in the matrix"; 25.7 works (`helma_h100.md`, section 2; `configure.sh:502-510`).
- **Orthogonality of cuSOLVER's** Q needed the Newton–Schulz step (Section 6).
- **Processes without MPS** are time-sliced and very slow (Section 9).

10.2 Intel

Toolchain. ifx with OpenMP offload to SPIR-V (`-fopenmp-targets=spir64`), oneMKL with its OpenMP offload interface for BLAS, and a C++/SYCL shim for LAPACK. On SuperMUC-NG Phase 2 only oneAPI 2025.1 works in production: 2025.3 compiles, but the OpenMP-to-SYCL interop crashes with the Level Zero V2 adapter (`intel_pvc_supermuc_ng2.md`; `GPU_PLAN_V2.md`, section 2).

Code and documents disagree. `CONSOLIDATION.md` records the verification of 2026-11-13 on the Max 1550 with ifx 2025.3, while the platform page says 2025.3 crashes at run time. `PLAN.md` and `DECISIONS.md` item 13 pin ifx 2026.2 for the test workstation. `NOT_IMPLEMENTED.md` (line 95) lists the question as open.

How BLAS and LAPACK reach the device. BLAS calls are written as ordinary Fortran calls preceded by `!$omp dispatch`; with `use onemkl_blas_omp_offload_lp64` ifx replaces the call by oneMKL's device variant when the arguments are mapped. LAPACK through `dispatch` would run partly on the host, so the factorisations go through a SYCL shim (`alf_la_sycl_shim.cpp`): the Fortran side creates an OpenMP interop object of type SYCL, extracts the `sycl::queue*`, and passes it with device addresses to C++ routines that call `oneapi::mkl::lapack::*` on that queue and wait. A probe checks that the OpenMP default device is the queue's device.

Problems and solutions.

- **Missing interop symbols.** ifx up to 2025.3 lacks `omp_get_interop_int/ptr` and the `omp_ipr_*` constants in `omp_lib`; they are declared with `bind(C)` and the OpenMP 5.1 values in `alf_la_mod.F90:142-145, 175-189`.
- **SYCL kernels in ifx-linked objects** are not found at run time; the SYCL code is built into shared libraries (`libalfsb.so`, `libalfdevqr.so`).
- **Unmapped arrays.** An unmapped array passed to the queue once hard-reset the Arc workstation; `need_present` checks the first and last element with `omp_target_is_present` before every queued call (`alf_la_mod.F90:290-306`).

- **Descriptor copies.** ifx copies the descriptors of module allocatables referenced in target regions at every launch; explicit-shape dummies cut the first-private arrays from 168 800 to 28 544 and launches from about 75 000 to 49 530 per sweep, worth 0.4 to 1.2 s at $L = 24$ (`report/sections/s6_performance.tex`, from `ISSUE_INTEL_VS_A30.md`).
- **Staging buffer.** The default Level Zero staging buffer made the 32 KB block copy cost 380 μs; a 1 MB buffer brings it to 22 μs (Section 7).
- **No asynchrony through OpenMP.** ifx runs `nowait` target tasks undeferred; asynchrony through SYCL on the interop queue (Section 7.9).
- **Hybrid oneMKL QR and slow `getrf`.** Own device QR (Section 6.8) and the LU-free phase (Section 6). `getrf` uses 64-bit pivots; a target loop converts them to the 32-bit pivots of the interface.
- **Ahead-of-time compilation.** A wrapper rewrites the target flag to `spir64_gen -device pvc`; start-up falls from about 65 s (just-in-time compilation of the kernels) to 14 s (`intel_pvc_supermuc_ng2.md`).
- **Faulty hardware.** All seven "Unstable matrix" aborts of a production series came from tile 1 of one node; the node is excluded and the precision threshold lowered (`DECISIONS.md`, item 46).

The optimisation steps, measured. Each step is a run-time switch of one binary, measured by alternating runs in one job (`ISSUE_INTEL_VS_A30.md`, section 2). Table 11.

Table 11: Max 1550, one chain per tile, seconds per sweep; per card with 4 ranks per tile and CCS 4. Mid-points of the measured ranges (`ISSUE_INTEL_VS_A30.md`, section 2; `SPEEDUPS.md`).

Step	Switch	$L = 24$	$L = 32$	sweeps/s per card
V0	none	10.6	31.0	0.549
V1	staging buffer 1 MB	7.07	23.2	0.568
V2	+ `ALFSH_ASYNCBLK=1`	6.56	22.2	0.67
V4	+ `ALFSH_DEVQR=1`	6.15	19.4	0.76
V7	+ `DEVQR=2`, `DEVUNMQR=1`	4.96	16.4	0.86
	+ parallel `acc0`	4.42–4.48		
	+ LU-free phase	4.04		

10.3 AMD

Toolchain. gfortran 14 with offload to `amdgcn-amdhsa` (`-march=gfx90a`); hipBLAS and hipSOLVER through a small C++ shim compiled with g++; own kernels compiled with `hipcc`. `-ffast-math` is removed for `GPU=amd`: the $L = 6$ test became unstable with it (`configure.sh:624-630`).

The launch cost of the GCC runtime. The OpenMP runtime of GCC (libgomp) launches kernels on AMD GPUs through its GCN plugin. An empty target region cost 253 μs, a target region with a parallel loop about 150 μs, against about 15 μs for a HIP launch plus synchronisation (`amd_mi210/README.md`; `alf_hip_kernels.hip.cpp:2`). With tens of thousands of launches per sweep this alone made the port useless. Three measures:

1. **A patched plugin** (`plugin-gcn-pool-prof.patch` for GCC 14.2.0): it pools the kernel argument block and the completion signal and small device allocations. An empty target region falls from 253 to 37 μs; about 3 times faster on the real run. `GCN_NUM_THREADS=4` instead of 16 was best.
2. **HIP kernels** for all small operations (`alf_hip_kernels.hip.cpp`, 621 lines, called through `alf_hipk_mod.F90`): block gather, scale, fused block end, flush gather, the observable kernels, the stabilisation helpers (row division, Newton–Schulz, phase, precision check, permutations),

the vertex slice multiplication, the column norms and permutations of `la_qrpp`. Under `#if defined(ALF_LA_HIP)` they replace OpenMP kernels in `Prog/gpu_*.F90`.

3. **Deferred synchronisation** (`plugin-gcn-presync-hook.patch`, `ALF_HIP_ASYNC`): normally every HIP call ends with `hipDeviceSynchronize`. With the patch the plugin calls a function registered by ALF before it launches an OpenMP kernel or copies data; only then are pending HIP operations synchronised. Output is bit-identical.

Together with the own QR (Section 6.8): $L = 16$, two sweeps, 66 s with stock GCC, 21 s with the plugin, threads and observable kernels, 6 s with the HIP kernels; $L = 24$ 95→19 s (`amd_mi210/README.md`). Steady state: 4.41 s per sweep at $L = 24$ and 17.5 s at $L = 32$.

Code and documents disagree. The comments in `alf_la_hip_mod.F90` (lines 5-6) and `alf_la_hip_shim.cpp` (lines 2-4) say every call ends with `hipDeviceSynchronize`; with the patched plugin this is no longer true for BLAS and own kernels. The `ALF_LA_TEST_*` hooks are declared but not armed in the HIP backend.

10.4 What would a fourth backend need

From the three experiences: an implementation of the `om_*` (or CUDA-style) routines of `alf_la`; a check that library calls and OpenMP kernels are ordered (the test `t_order`); a measurement of the launch cost of OpenMP target regions, and if it exceeds about 20 μs, native kernels for the small operations; a measurement of the vendor QR, and an own QR if it is slow or hybrid. An Apple GPU was evaluated and rejected: Metal has no double precision; emulated double-double ZGEMM reached 290 GFlop/s against 460 to 520 GFlop/s of the CPU's matrix unit, and inside ALF at $L = 24$ it saved 0.6 s of 115 s while worsening the precision of G from 1.6×10^{-12} to 6.7×10^{-11} (`APPLE_GPU_FEASIBILITY.md`).

11 Verification

A Monte Carlo code is hard to test: the output is noisy, the trajectory is chaotic, and the random number stream differs between compilers. The port went through two verification regimes. The first relied on statistical acceptance gates and was abandoned; the second relies on deterministic comparisons and is the standard now. Both are described, because the contrast is the main lesson.

11.1 The method that worked

1. Same-compiler CPU reference at fixed seed. The host draws the same random numbers in the same order as the CPU code, and makes the same accept/reject decisions from the same ratios. If the device computes G to rounding accuracy, a GPU build and a `GPU=none` build of the same compiler with the same parameters and seed follow the same Markov chain. A wrong decision would occur only if a ratio lies within rounding of a random number, which over a short run does not happen. The observables then agree to rounding: about 10^{-13} with `Mz=.F.` and about 10^{-10} with `Mz=.T.`, where the coupling is complex and the trajectory more sensitive (`README.md`, "How correctness is checked"). The comparison script is `scripts/defect/run_cmp.py`. The same compiler matters: random streams differ between gfortran, nvfortran, ifx, so trajectories cannot be compared across compilers.

The rule for large lattices was refined on 2026-11-11 (`DECISIONS.md`, item 43): for $L \geq 16$ on the Max 1550, 10^{-13} is not reachable in the imaginary parts of the scalar observables because of the summation order. The rule now is: configurations (`confout`) bitwise identical; real parts within 10 times the spread between two CPU runs; imaginary parts of the scalars reported only. The bitwise identity of the configurations is the strongest test: it means every one of the millions of decisions was the same.

2. Deterministic kernel tests. Each device routine has a self-test at fixed fields that compares with the host routine on the same input: the mirror round trip and the wraps (`ALF_GPU_SELFTEST`=1, 2), the stabilisation (3), the τ pass (4), the update (`ALF_GPU_UPDTEST`), plus unit tests of `alf_la` (`t_order`, `t_info`, `t_qrpp`).

3. Bitwise GPU-versus-GPU checks. After ticket 212 every reduction runs in a fixed order, so two GPU runs with the same input are bitwise identical. A switch that should not change the numerics (the staging buffer, the asynchronous block path, MPS, CU masking) is accepted only if the output is bitwise identical with and without it.

4. Planted defects. Every check is shown to fail on a deliberately wrong program before it is trusted: the fault switches (`ALF_GPU_FAULT`, `FAULT_STAB`, `FAULT_TAU` with values such as `perm`, `phase`, `eps:1e-12`) inject a known error. For example, perturbations of 10^{-11} and 10^{-12} fail the decision-43 rule as they must; the pre-fix device QR library, used as a planted defect, revealed a race that appeared only with four ranks per tile under CCS (`EVAL_ORIGINAL_GPU_BRANCH.md`, section 4).

5. A production guard. In a production run any debug or fault variable stops the run (`gpu_state_mod.F90:412-421`): "GPU debug env refused: ... (production run; set ALF_GPU_DEBUG=1 for a debug run)". A fault switch left in a job script cannot silently corrupt data.

6. Refusal at setup. What is not supported is refused before the run, with a reason in the `info` file (Section 13).

11.2 What did not work, and why

The first phase of the port (tickets 201 to 208, October 2026) required, for each ticket, a pre-registered statistical acceptance test with positive and null controls, tiers, an independent verifier and evidence archives (`VERIFICATION.md`). It failed in practice (`EVAL_ORIGINAL_GPU_BRANCH.md`, sections 3-4; `DECISIONS.md`):

- **The "sd condition"** of ticket 207 (ratio of standard deviations GPU/CPU at most 1.25) failed on *correct* CPU variants, with values 0.9 to 2.1.
- **The statistical comparison `compare2`** detected a planted shift of 5 standard errors only about 60 % of the time; at 8 standard errors about 99 %. A correct implementation failed about 33 % of the time by chance, and the false-alarm budget was raised from 0.04 to 0.35. Twice the result was inconclusive and the run length was raised. Its minimal detectable effect at 80 % power was 0.47 % in the energy, 18 % in $S_{zz}(\pi,\pi)$ and 27 % in χ_{zz}: "a gross-error detector" (`BASELINE.md`).
- **Tolerances at rounding level as maxima**: the requirement that the maximal τ precision be within 1.5 times the CPU's failed with ratios 1.8 to 2.1 on identical trajectories. The maximum of a rounding-driven quantity over a run is an extreme-value statistic and fluctuates (`DISCREPANCIES.md`, 2026-10-29). Similar: the `STABLOG` same-seed deviation 1.214×10^{-9} against a bound of 10^{-9}; later 1.36×10^{-10} against 10^{-10} at $L = 6$, `Mz=.T.` on Intel, where oneMKL itself fails 2 of 5 seed sets.
- **A control that could not be killed**: switching off the Newton–Schulz step on oneMKL did not make the precision test fail, because oneMKL's Q does not need the step; it was replaced by a graded fault (`DECISIONS.md`, items 24-27).

The cost: verification infrastructure, process and documents were 62 % of the 52 632 added lines; 172 of 208 commits touched no code; one acceptance test was pre-registered seven times; ticket 207 stalled for about 4.5 days (`EVAL_ORIGINAL_GPU_BRANCH.md`, section 3). The key finding (section 4 of the same file): every bug in the port code was caught by a cheap deterministic check (a kernel test against the host, a unit test, the same-seed comparison, or a determinism test). No port bug

was caught by a statistical gate; the statistical harness found only bugs in itself. On 2026-11-02 the owner closed the process ("If it works it works ... We will not make any stricter tests", `DECISIONS.md`, item 41). The next ticket took 9 hours; in about seven further days the delayed and submatrix updates, time-dependent hopping, MPI, parameter-parallel runs, the H100, the Max 1550 production with own QR, and the AMD backend followed.

Why statistical gates are weak here: the GPU and CPU runs at the same seed are not independent samples; at different seeds the comparison has the power of the shorter run and can only detect gross errors. Bounds at rounding level are limited by rounding noise, not by the correctness of the code. A test of identity of trajectories is both stronger and cheaper.

11.3 Current results per backend

Table 12: Verification of the unified branch (`CONSOLIDATION.md`, 2026-11-13, unless stated). Deviations are the largest relative deviation of the observables against the same-compiler CPU build at the same seed.

Backend	Result
`GPU=none`	bitwise identical to the pre-merge tip `cc473abc`; bitwise identical to upstream ALF with `LOBS_ST=1`
AMD MI210	1.15×10^{-14} ($L = 6$), 5.0×10^{-14} ($L = 12$), acceptance identical; bitwise identical to the pre-merge AMD tip at $L = 6, 12, 16, 24$
Intel Max 1550 (ifx 2025.3)	default switches: 2.2×10^{-15} ($L = 6$, `Mz=.F.`), 1.5×10^{-14} ($L = 12$), 5.5×10^{-11} ($L = 6$, `Mz=.T.`); V7: 2.3×10^{-15}, 1.6×10^{-14}, 1.36×10^{-10} (tolerance 10^{-10}); LU-free phase equal to the LU phase to 5.8×10^{-15}; $L = 16, 24$ pass the decision-43 rule
NVIDIA H100	5.2×10^{-13} at $L = 8$, acceptance identical (0.3377294921875); restart continues bitwise (`helma_h100.md`, section 3); the row of the unified commit is a placeholder in `CONSOLIDATION.md` **[TODO: H100 check of the unified commit]**
NVIDIA A30	unified build compiled, GPU run not done (card occupied) **[TODO: A30 check of the unified commit]**
Delayed, submatrix	reproduce the standard update to 10^{-12}; `SpinZ_localtau` agrees with the exact $U = 0$ result to 4.7×10^{-14}

In production the precision of G (maximum over a run) lies between 10^{-10} and 3×10^{-8} up to $L = 44$; a corrupted G from faulty hardware gave 10 to 28 (`DECISIONS.md`, item 46).

Code and documents disagree. `GPU_PLAN_V2.md` (section 1) states that no same-seed comparison of AMD against the CPU was found and lists it as work package WP1.3; `CONSOLIDATION.md` reports it (1.15×10^{-14} and 5.0×10^{-14}). `VERIFICATION.md` (section 7) still says the default path is not bitwise reproducible; ticket 212 fixed that.

Open items. An Intel check on the Arc B580 with `Ltau=1` showed a 6.3σ difference in $S_{zz}(\pi, \pi)$, $S_{zz}(\pi, \pi, \beta/2)$ and χ_{zz}; whether this is a fluke or a bug is open (`EVAL_ORIGINAL_GPU_BRANCH.md`, section 2). Ticket 213 (the `STABLOG` same-seed excess) was never started.

12 Performance

All numbers are for the production model unless stated: Hubbard model with the space-time point defect, `Mz=.F.`, $N_{\mathrm{FL}} = 2$, $N_{\mathrm{SUN}} = 2$, $U = 5$, $h_0 = 1$, `Symm=.T.`, $\Delta\tau = 0.1$, $\beta = L$ (so $L_{\mathrm{trot}} = 10L$), `Ltau=0`, `Nwrap`=10, submatrix update. One sweep goes up and down through all slices and includes the measurements. The short report (Section 6 of `gpu_port_report.tex`) states how each number was obtained; the sources are given here in comments. The numbers come from different sessions and timers; the short report lists the differences.

12.1 Elementary operations

Table 13 gives the Phase 1 kernel benchmarks on the A30 against one host core. They explain the design: dense products and QR gain 37 to 140 times at $N = 1024$, the rank-one update gains little, and a rank-k update costs 25 to 40 times less per update than rank one.

Table 13: A30 kernel benchmarks in μs, device and one host core (`BENCHMARKS.md`, Phase 1).

	$N = 144$		$N = 400$		$N = 1024$	
Operation	device	host	device	host	device	host
Dense B_T product	53	1095	172	22 941	2757	388 561
Stabilisation QR	1058	1834	2822	25 721	9956	373 241
Rank-1 update	4.7	6.8	6.6	71	31.6	353
Rank-k ($k = 128$), per update	0.21		0.37		1.25	

12.2 Time per sweep against L

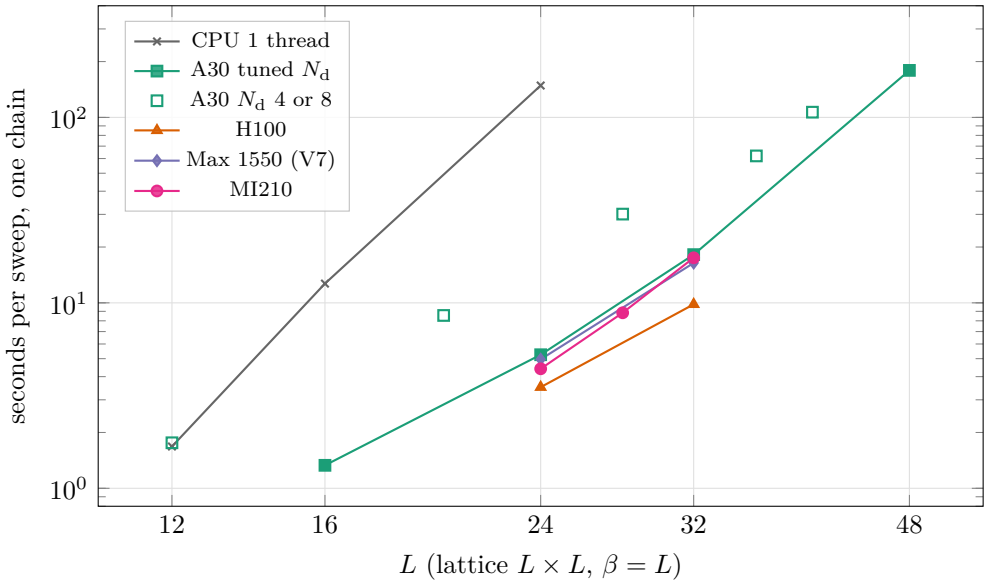


Figure 3: Seconds per sweep for one chain. Sources: CPU and A30 `TUNING_DEFECT.md`; H100 `helma_h100.md`; Max 1550 `ISSUE_INTEL_VS_A30.md` (V7, before the `acc0` and LU-free changes); MI210 `amd_mi210/README.md`. Open squares: A30 with N_d not tuned.

Table 14: A30, one chain (`TUNING_DEFECT.md`, table "Time per sweep, beta = L"). Peak device memory and the precision of G (mean / max over the run).

L	N	L_{trot}	N_d	GPU s	CPU 1 thread s	memory MiB	precision
12	144	120	4	1.76	1.68	299	1e-14 / 7e-13
16	256	160	32	1.33	12.7	385	2e-14 / 3e-12
20	400	200	4	8.55		619	5e-14 / 5e-11
24	576	240	32	5.25	148.6	1069	4e-14 / 1e-11
28	784	280	4	30.1		1859	6e-14 / 3e-11
32	1024	320	64	18.2	≈860 (extrap.)	3077	6e-14 / 6e-11
36	1296	360	8	62.0		5189	2e-13 / 9e-11
40	1600	400	8	106.7		8391	2e-13 / 2e-10
48	2304	480	32	179.3		19 647	6e-13 / 7e-10

Scaling. The work per sweep grows as $N^3 L_{\mathrm{trot}} \propto L^7$ for the cubic parts (wraps with dense hopping, stabilisation, `Symm`), as $N^2 L_{\mathrm{trot}} \cdot N/N_{\mathrm{d}} \propto L^7/N_{\mathrm{d}}$ for the flushes, and as $N L_{\mathrm{trot}} \propto L^3$ for the latency-bound per-block work. Fits $t \propto L^b$ give $b = 6.44$ for one CPU thread, 4.45 for the A30 (tuned points $L = 16$ to 48, local exponents 3.4, 4.3 and 5.6), 3.58 for the H100 and 4.17 for the Max 1550 (two points each), and 4.78 for the MI210 (`report/sections/s6_performance.tex`, Table "fits"). The GPU exponents are well below 7 because at these sizes the device is not yet saturated by the cubic work: the latency-bound part ($\propto L^3$) still matters, and the local exponent rises with L towards the asymptotic value (5.6 between $L = 32$ and 48 on the A30).

Comparison with the CPU. Against one CPU thread of the same build the A30 is 1.0 times as fast at $L = 12$, 9.6 times at $L = 16$, 28 times at $L = 24$ and about 47 times at $L = 32$ (the last against an extrapolation from four threads) (`TUNING_DEFECT.md`). This is not the fair comparison. A GPU replaces the cores it displaces: 16 Zen 3 cores per A30 on nividia, 14 Sapphire Rapids cores per Max 1550 tile on SuperMUC (`HARD.md`). With perfect scaling over independent chains, 16 cores would give 16 times one thread; then the A30 at $L = 24$ (one chain, 5.25 s) is about 1.8 times a 16-core node, and with several chains per GPU more. A measured node-level CPU number for the defect model does not exist **[TODO: CPU node throughput for the defect model at L=24]**. For the plain Hubbard model a 16-rank node delivered 26 chain-sweeps per second at 12×12 and 1.8 at 20×20 with checkerboard hopping (`BENCHMARKS.md`, Phase 0); these are not comparable with the defect model, whose hopping is dense.

Small lattices. At $L = 12$ the GPU gains nothing. At $L = 32$ the A30 still issued 175 000 small kernel launches and 220 000 stream synchronisations per sweep; below $L = 16$ the GPU is mostly waiting for the host (`TUNING_DEFECT.md`).

12.3 Update scheme and N_{d}

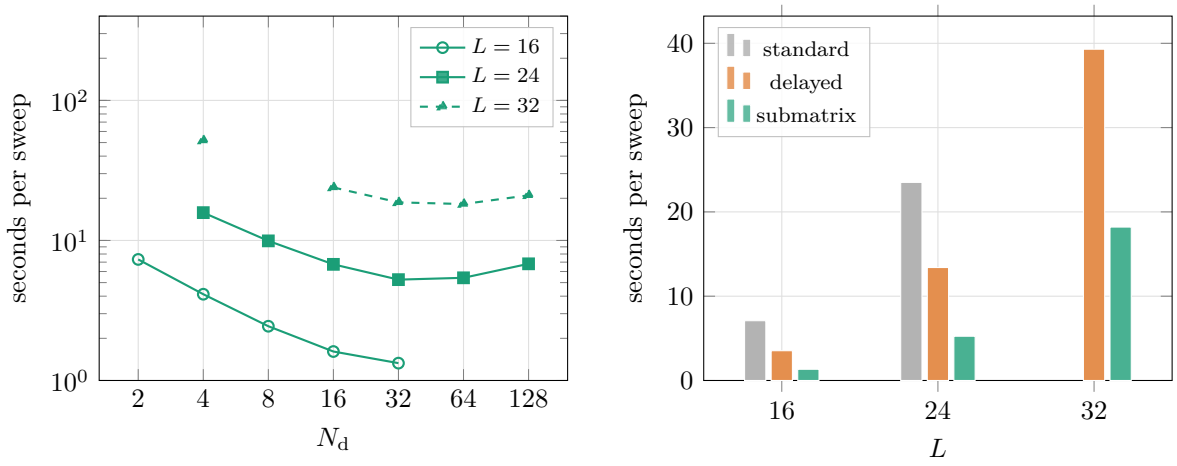


Figure 4: A30, $\beta = L$. Left: submatrix scheme against N_{d}. Right: the three schemes, delayed and submatrix at their best measured N_{d}; standard not measured at $L = 32$ (`TUNING_DEFECT.md`, table "Scheme and N_delay").

The minimum in N_{d} (Figure 4) is the balance of two costs. Per block there is a fixed cost of about ten device operations and one round trip, so a small N_{d} is latency-bound: halving N_{d} from 32 to 16 doubles the number of blocks and costs 21 to 29 %. A large N_{d} increases the host work per proposal, $O(k^2)$ for the small matrix M with k up to N_{d}, and the cost of the flush GEMMs grows with k while the number of blocks per slice cannot fall below one per N_{d} vertices. Above $N_{\mathrm{d}} = 64$ the time rises again (6.81 s at 128 against 5.25 s at 32 for $L = 24$). The recommendation is $N_{\mathrm{d}} = 32$ for $L \leq 28$ and 64 for $L \geq 32$; the automatic rule chooses 64 at $L = 32$ (`TUNING_DEFECT.md`). With several ranks per GPU the optimum shifts upwards (H100, 8 ranks, $L = 32$:

66.5 s at $N_d = 32$, 34.5 s at 64, 34.3 s at 128, `helma_h100.md`, section 4.2). On Intel the ranking of the schemes differs: delayed was slower than standard there (19.66 against 18.32 s per sweep, `p7/NOTES.md`, lines 105-106).

12.4 `Nwrap`

Table 15: A30, $L = 24$, submatrix $N_d = 8$ (`TUNING_DEFECT.md`, table "Nwrap").

`Nwrap`	s/sweep	memory MiB	precision mean / max
5	11.8	1553	2e-14 / 9e-12
10	9.9	1069	4e-14 / 1e-11
20	9.2	825	3e-13 / 5e-11
40	8.7	703	6e-11 / 1.6e-8

Halving `Nwrap` doubles the number of decompositions and `CGR` calls and the size of the stack. At $L = 24$ `Nwrap`=5 costs 19 % more time and 45 % more memory than 10; 20 saves 7 % at a precision of 5×10^{-11}; 40 saves 12 % but the precision drops to 1.6×10^{-8} (Table 15). With `Mz=.T.` at $U = 5$, `Nwrap`=10 is not sufficient (precision maximum 2×10^{-3}); 4 to 5 are needed (`TUNING_DEFECT.md`).

12.5 Where the time goes

One chain. On the A30 at $L = 32$ ($\beta = 4$, nsys profile), 44 % of the kernel time is in 32 338 small flush ZGEMMs of 52 μs, 17 % in dense GEMMs, about 9 % in QR and LU; the kernels run 3.8 s of a 7.5 s sweep, and 65 % of the host API time is `cuStreamSynchronize` (`TUNING_DEFECT.md`). On the Max 1550 at $L = 24$ (V7) the phases are updates 1.44 s, stabilisation 0.80 s, `CGR` 1.21 s, observables 1.18 s (`p7/NOTES.md`, section 3); the largest device items are the device QR (192 calls, 0.82 s), oneMKL `getrf` (96 calls, 0.38 s, since removed by the LU-free phase), the 576^3 hopping GEMMs (2880 calls, 0.32 s) and the rank-k flushes (34 560 calls, 0.31 s). On the MI210 the QR panel is 36 % of the kernel time.

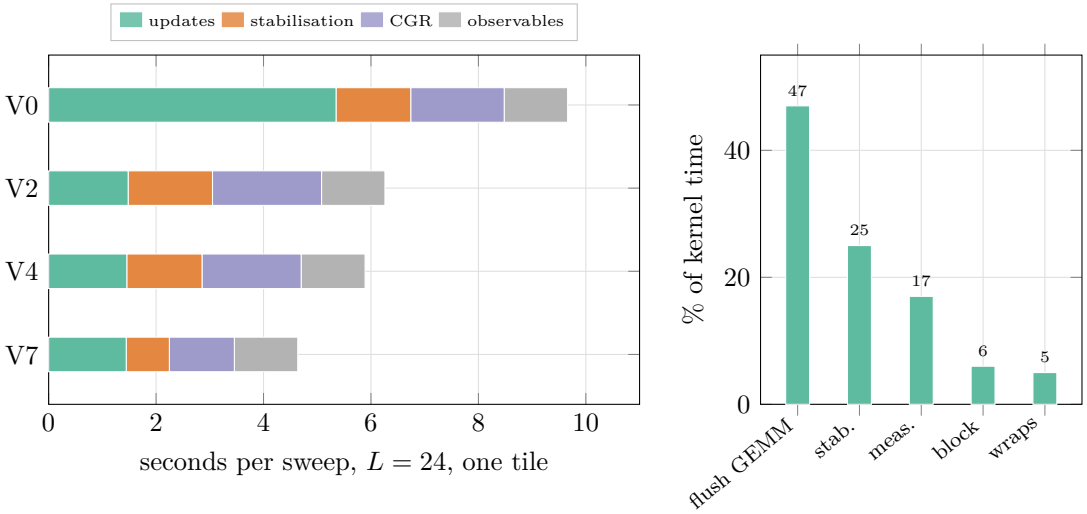


Figure 5: Left: Max 1550, $L = 24$, one chain per tile, wall time per sweep by phase for the variants of Table 11 (`ISSUE_INTEL_VS_A30.md`, section 3; `p7/NOTES.md`, section 3). The update phase falls with the staging buffer (V0→V1, not shown separately) and the asynchronous path; stabilisation and `CGR` fall with the own QR (V4, V7). Right: A30, 9 ranks under MPS, $L = 24$, share of the kernel time of one rank (`PP_GPU.md`).

Production layouts. With nine ranks on the A30 under MPS a kernel runs 99.97 % of the time, 5.6 kernels at once on average; the kernel time splits into flush GEMMs 47 %, stabilisation 25 %, measurements 17 %, block kernels 6 %, wraps 5 % (`PP_GPU.md`). Here the device is full, and only a reduction of device work helps. "A change that removes launches helps single chains; only a change that removes device work helps production" (`SPEEDUPS.md`).

12.6 Counting the operations of one sweep

The structure of Sections 5 to 8 predicts how often each device operation runs. At $L = 24$ ($N = 576$, $L_{\mathrm{trot}} = 240$, `Nwrap`=10, $N_{\mathrm{d}} = 32$, $N_{\mathrm{FL}} = 2$) one sweep visits $2L_{\mathrm{trot}} = 480$ slices and has $N_{\mathrm{ST}} = 24$ stabilisation points per direction. The prediction and the counts measured on the Max 1550 (`p7/NOTES.md`, sections 2 and 6) agree:

Table 16: Device operations per sweep at $L = 24$: count from the algorithm and measured count (`p7/NOTES.md`).

Operation	Derivation	Predicted	Measured
N^3 GEMM, hopping wraps	480×2 flavours $\times 2$ (G) + $480 \times 2 \times 1$ (`WRAPUR/UL`)	2 880	2 880
N^3 GEMM, `Symm` for measurements	480×2 flavours $\times 2$	1 920	1 920
QR (`geqrf`)	$(24 + 24) \times 2$ decompose + same for `CGR`	192	192
LU (`getrf`, before the LU-free phase)	one per `CGR`	96	96
blocks	$480 \times \lceil 576/32 \rceil$	8 640	8 640
flush gathers	8640×2 flavours	17 280	17 280
rank-k GEMM	8640×2 flavours $\times 2$	34 560	34 560
`corr` kernels	one per measurement and kind group	480	480

The measured device times per item (Max 1550, V7, in isolation) then give the budget: QR 0.82 s, `getrf` 0.38 s, hopping GEMMs 0.32 s, rank-k GEMMs 0.31 s, `corr` 0.47 s, `unmqr` 0.26 s, `Symm` GEMMs 0.21 s (`report/sections/s6_performance.tex`, Table "p7top"). A 576^3 ZGEMM takes about 109 μs; at $8 \cdot 576^3 = 1.53$ GFlop this is 14 TFlop/s, so the dense part runs at a rate typical of large matrix products on this device. A rank-k GEMM of $576 \times 576 \times k$ with $k \approx 12$ takes 9 μs for $8 \cdot 576^2 \cdot 12 \approx 0.032$ GFlop, about 3.5 TFlop/s: it is bound by reading and writing G (2×5.3 MB in 9 μs, about 1.2 TB/s). The 8 640 gathers, scales and copies are each a few microseconds of device time but are submitted one by one; their submission cost dominates the update phase (Section 7).

12.7 Throughput per device

Table 17: $L = 24$, one chain and best measured layout (`report/data/platform_*.dat` and sources there).

Device	one chain s/sweep	sweeps/s per device	layout
A30	5.25	0.44	8 ranks
H100	3.51	1.176	8 ranks, MPS 25 %
Max 1550 (card = 2 tiles)	4.04–4.96 per tile	0.86	4 ranks per tile, CCS 4 (V7)
MI210	4.41	0.434	4 processes

12.8 Tuning rules

From the measurements (`TUNING_DEFECT.md`, `helma_h100.md`, `RUNTIME_SWITCHES.md`):

- `Sequential_scheme="submatrix"`; $N_{\mathrm{d}} = 32$ for $L \leq 28$, 64 for $L \geq 32$ (higher with many ranks per GPU).

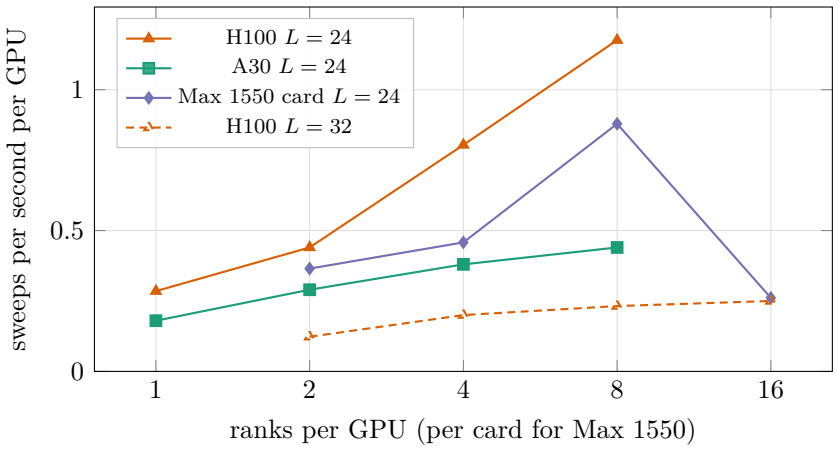


Figure 6: Aggregate throughput against ranks per device (`helma_h100.md`; `ISSUE_INTEL_VS_A30.md`, section 7).

- `Nwrap`=10 for `Mz=.F.`; 4 to 5 for `Mz=.T.`.
- `Ltau=0`: `SpinZ_localtau` comes from the equal-time pass.
- `OMP_TARGET_OFFLOAD=MANDATORY`, so that a failed device initialisation stops the run instead of running on the CPU.
- NVIDIA: MPS whenever more than one rank shares a GPU; 8 ranks per H100.
- Intel: `LIBOMPTARGET_LEVEL_ZERO_STAGING_BUFFER_SIZE=1048576`; `ALFSH_ASYNCBLK=1`; CCS 4 with 4 ranks per tile; ahead-of-time build.
- AMD: the patched GCC plugin, `GCN_NUM_THREADS=4`; 4 processes per GPU.
- Check the `info` file for "GPU path (gpu_state): ACTIVE" and "device equal-time measurement ACTIVE".

13 What is not ported, and what it would take

13.1 The refusal list

`gpu_supported` (`gpu_state_mod.F90:499-617`) checks, in this order, and on the first failure writes "GPU path (gpu_state): INACTIVE, reason: ..." into `info`; the run then continues on the CPU, unless `ALF_GPU_SELFTEST` is set, in which case it stops. The checks are on the structure of the model, not on its name.

1. `ALF_GPU_DISABLE=1`.
2. Tempering with exchange (`TEMPERING` without `PARALLEL_PARAMS`).
3. Stabilisation schemes `STAB1`, `STAB2`, `STAB3` (only the default and `STABLOG` are ported).
4. Projector code ($T = 0$).
5. Langevin and HMC updates.
6. Global moves in space and time (`Global_moves`, `Global_tau_moves`, `N_Global_tau`> 0).
7. `Propose_S0`.
8. MALA updates.
9. `Lowrank_cross_slice` (delay across slices).
10. `Calc_Fl` with weight reconstruction.
11. No OpenMP offload device.
12. Per vertex $\mathrm{Op}_V(n, f)$: not diagonal; field type other than 1 or 2; field type differing between flavours; $N_{\text{non-zero}} > 1$; a time-dependent coupling g_t.
13. Per hopping element: unknown type; time-dependent with `size(g_t)`$\neq L_{\text{trot}}$; time-dependent matrices above 2 GB; a static element after a time-dependent one.

In addition, after activation, the run stops for a `Sequential_scheme` other than standard, delayed or submatrix, and for a NaN scale with the default stabilisation ("Switch to LOG is required"). Device observables exist only for the Hubbard model; any other model measures on the host after a download of G. The Hubbard `Ham_Set` refuses the pinning field with the projector, Langevin/HMC or a non-default measurement window, in all builds, because of known bugs (`CONSOLIDATION.md`).

In practice this covers the predefined Hubbard model with Ising or four-valued fields and its variants with time-dependent hopping. Stock models such as Kondo, t-V or Spin–Peierls have vertices with $N_{\text{non-zero}} > 1$ or non-diagonal operators and are refused (`NOT_IMPLEMENTED.md`, lines 45-47).

13.2 What each item would take

The estimates are those of `GPU_PLAN_V2.md` (work package 3; S $<$ 2 days, M 2–10 days, L $>$ 2 weeks), with the technical reason from the code.

- **General vertices (rank r, non-diagonal).** A vertex acting on r sites changes G by a rank-r update with an $r \times r$ ratio matrix. The wrap becomes a product with an $r \times r$ block on rows and columns P instead of a scaling. Work started in ticket 401-A (`Calc_Fl` on the device, general vertices, exact-diagonalisation references for Plain_Vanilla, t-V, Kondo, Spin–Peierls) and was stopped on 2026-11-07; it is not in the unified branch and is "the main unfinished item of value" (`EVAL_ORIGINAL_GPU_BRANCH.md`). Rebasing it: M; general rank-r vertices in the standard scheme: M–L; delayed/submatrix for rank r: M.
- **Observables of other models.** A host fallback (download G per measurement, call `Obser`/`ObserT`) is M and works for any model, at the cost of $2L_{\text{trot}}$ transfers of $16N^2N_{\text{FL}}$ bytes per sweep. Device versions need one kernel per observable, model by model.
- **Tempering exchange.** After a swap the device mirror (fields, possibly parameters) must be refreshed: S–M.
- **Global moves and `Propose_S0`.** A global move needs G recomputed for the proposed configuration (a full stabilised sweep segment on the device) and a ratio of determinants: M–L.
- **Continuous fields, Langevin, HMC, MALA.** Field types 3 and 4 need new exponential tables; Langevin and HMC need forces $\partial S/\partial\phi$, which are traces of G with vertex derivatives, as new device kernels: L.
- **Projector ($T = 0$).** New routines for the trial wave function and `CGRP`, and the projector τ pass: L.
- **`STAB1/2/3`, cross-slice delay**: proposed as permanent refusals unless a user needs them.
- **`STABLOG` fully on the device** would need a GPU QR with column pivoting that meets the `STABLOG` precision; none was found (Section 6).

13.3 Ranked speedup candidates

`SPEEDUPS.md` (state 2026-11-13) separates single-chain latency from production throughput. Available without code changes: the Intel staging buffer, the asynchronous block path, `ALF_GPU_OBS_SKIP` (7 % under MPS), `Nwrap`=10 instead of 5 (5 costs about 45 %), the Intel parallel `acc0` reduction (about 10 %). Implemented but not measured on NVIDIA: the LU-free phase (measured on Intel, 4.45→4.04 s), removal of redundant `la_sync` calls (10–20 ms per sweep).

The larger candidates, ranked by expected effect on production throughput (`SPEEDUPS.md`, section 4; `GPU_PLAN_V2.md`, WP4):

1. **Two-level submatrix update.** One flush per slice of rank about 190 instead of one per block with rank 10 to 21, with the $k \times k$ algebra on the device. Targets the flush GEMMs, 47 % of the kernel time in production on the A30. Effort L.

2. **FFT or band form of the static hopping.** The static hopping of the defect model is translation invariant; an FFT would replace about 5800 dense GEMMs per sweep at $L = 24$. Needs an FFT per backend and a demonstration that truncation stays below one unit of rounding. M–L.
3. **Coalesced `corr` kernel.** The Intel correlation kernels are 3.5 times slower than their data volume implies; a tiled transpose would fix the memory access pattern. M.
4. **Measure on every second slice.** Halves the 12 to 25 % of a sweep spent in measurements; changes the estimator's autocorrelation, so it is a physics decision.
5. **QR panel look-ahead** (Intel, AMD): overlap the next panel with the trailing update; single-chain gain. M.
6. **Device `getrf` on Intel**: largely superseded by the LU-free phase.

Measured and rejected: a CUDA port of the AMD QR (cuSOLVER QR and LU are about 8 % of a sweep), a fused block-end kernel (no gain), $N_\mathrm{d} = 128$, `Nwrap`≥ 20, oneMKL batched routines, `nowait` on ifx, CholeskyQR2, `STABLOG` for speed. Batching several walkers in one process has a realistic ceiling of 1.3 to 1.6 times for 15 to 20 person-days in own kernels only; a full rebuild is not recommended (`ISSUE_INTEL_VS_A30.md`, section 8).

14 Lessons for a GPU port of the whole of ALF

`GPU_PLAN_V2.md` (2026-11-13) records what the first plan assumed and what the measurements showed. The lessons below are taken from it, with the evidence from this document.

1. **OpenMP offload is a good base, not a complete solution.** The bulk kernels and the data management in OpenMP compiled and ran on three vendors from one Fortran source. But each vendor needed a small native layer for the hot small operations: SYCL on Intel for asynchrony, HIP on AMD for launch cost. Plan for it behind the library boundary from the start (Section 10).
2. **Keep one library boundary.** `alf_la` with a host reference implementation made every backend testable against LAPACK, and kept vendor code out of the physics. New device operations should enter as `la_*` routines with a host version.
3. **Measure each primitive before redesigning.** The assumed cost of a synchronisation on the Max 1550 (100 to 170 μs) was wrong (6 to 18 μs); the real cost was a 32 KB copy through a small staging buffer (380 μs, 22 μs with a larger buffer). "One environment variable gained more than two code phases."
4. **Algorithm before kernels.** The first plan put the delayed and submatrix updates into a late phase and expected the stabilisation to dominate. The rank-one update turned out to be the largest kernel, and the submatrix update in block mode became the production scheme, 2.4 to 5 times faster. The next large gain is again algorithmic (two-level submatrix update).
5. **Vendor solver quality varies; decide per routine by measurement.** cuSOLVER was adequate; oneMKL's QR was hybrid and rocSOLVER's launch-bound. Own QR kernels of a few hundred lines fixed both. The orthogonality of cuSOLVER's Q needed a Newton–Schulz step.
6. **The model must be flattened.** Derived types with allocatable components and polymorphic classes do not go to the device. A flat mirror built once at setup, plus refusal of everything the mirror cannot express, kept the CPU code untouched. A port of all of ALF needs a mirror for general vertices (rank r, non-diagonal, continuous fields) and for the observables of each model, or a host fallback for observables.
7. **Refuse rather than partially offload.** A feature is either fully on the device or refused at setup with a reason. Partial residency multiplies transfers and makes the data flow hard to reason about.
8. **Keep the random numbers and the decisions on the host.** It costs a few bytes per

block and makes same-seed comparison with the CPU possible, which is the strongest and cheapest verification there is.

9. **Make reductions deterministic.** Fixed-order reductions make two GPU runs bitwise identical, which turns every switch and refactoring into a bitwise test.
10. **Rounding-level tolerances and statistical gates are weak tests.** They were costly, noise-limited, and caught no port bug. Bitwise identity and same-seed trajectories decide; statistics are reported. Every check must first be shown to fail on a planted defect, and verification should cost a bounded share of the work (at most about 20 %).
11. **Single-chain speed and throughput are different targets.** One chain is latency-bound; a production layout with several chains per GPU is throughput-bound. A change that removes launches helps the first; only a change that removes device work helps the second. The production layout (ranks per GPU, MPS share, CCS, N_d) is a measured per-platform setting.
12. **Toolchains are fragile.** On the Max 1550 only oneAPI 2025.1 worked; nvhpc 24.11 produced NaN; nvfortran miscompiled one routine at `-O2`; GCC's AMD runtime needed a patch. Keep per-platform pages with a tested toolchain, and recheck on every module change.
13. **Hardware fails.** A faulty GPU tile produced corrupted Green functions. A tight precision threshold with an automatic restart, and a short health test per tile before production, are part of a GPU production setup.

For an upstream merge `GPU_PLAN_V2.md` (WP5) proposes an 11-patch series, each building and passing the CPU tests with `GPU=none` unchanged: `alf_la` with the host backend; the configure switch; the CPU delayed/submatrix updates; `gpu_state_mod` with `gpu_supported` and the production guard; wraps and stabilisation with CUDA; updates; observables and τ pass; the oneMKL backend; the HIP backend; time-dependent hopping; user documentation.

A Glossary of GPU terms for a QMC physicist

Host, device. The CPU with its memory, and the GPU with its own memory. They are connected by a bus (PCIe or a faster link). Data must be copied explicitly between the two memories (or migrated by the driver with unified memory, which the port does not use).

Transfer, D→H, H→D. A copy between device and host memory. It costs a fixed latency (a few μs) plus the size divided by the bandwidth (10 to 50 GB/s). An $N \times N$ complex matrix at $N = 576$ is 5.3 MB.

Kernel. A function that runs on the device, executed by many threads in parallel. In OpenMP it is the body of a `target` region.

Launch, launch latency. Starting a kernel. The host sends a command to the device; this costs several μs (Table 8) regardless of the size of the kernel. Many small kernels are therefore slow.

Synchronisation. The host waits until the device has finished submitted work. A *synchronous* operation returns only when complete; an *asynchronous* one returns at once, and the host can prepare the next operation while the device works.

Stream, queue. An ordered list of device operations (CUDA: stream; SYCL: queue). Operations in one stream run in order; operations in different streams may overlap. An *in-order* SYCL queue behaves like a CUDA stream.

Thread, team, block, work-group, wave. The device runs threads in groups that can share fast local memory and synchronise among themselves: a *team* (OpenMP), *block* (CUDA/HIP) or *work-group* (SYCL). Within a group, 32 or 64 threads (a *warp* or *wave*; on Intel a *sub-group* of 16) execute the same instruction together.

`target teams distribute parallel do`. The OpenMP construct that runs a loop on the device: iterations are distributed over teams, and within a team over threads.

`target enter data`, `target update`. OpenMP directives that create a device copy of a host array (with or without copying the values) and copy values between the two copies.

`use_device_addr`. OpenMP clause that gives host code the device address of a mapped array, to pass to a library.

Interop. OpenMP mechanism (`!$omp interop`) to obtain the native stream or queue that the OpenMP runtime uses, so that library calls can be ordered with OpenMP kernels.

Reduction. Combining many values into one (sum, maximum). On a device it is done as a tree of partial sums. An *atomic* addition lets many threads add into one location safely, but in a non-deterministic order.

Shared/local memory, registers. Small fast memory per team (tens of KB) and per thread. Kernels that keep data there avoid the latency of the main device memory (about 1 μs per dependent access on the Max 1550).

Occupancy, utilisation. The fraction of the device's compute units that have work. A single small matrix product uses a fraction of a large GPU.

Memory bound, compute bound. A kernel limited by the speed of reading and writing memory (rank-one update: 32 bytes per 8 flops) or by arithmetic (large matrix product: $16N^3$ flops for $48N^2$ bytes).

FP64. Double precision. Data-centre GPUs (A30, H100, Max 1550, MI210) have fast FP64; consumer and Apple GPUs do not.

cuBLAS, cuSOLVER, oneMKL, hipBLAS, hipSOLVER, rocSOLVER. Vendor libraries for BLAS and LAPACK on NVIDIA, Intel and AMD devices.

Hybrid routine. A library routine that runs part of its work on the host CPU (oneMKL's `geqrf` factorises panels on the host).

Panel, trailing update, compact WY. A blocked QR factorises a narrow block of columns (the panel) with Householder reflectors, then applies them to the remaining columns (the trailing update). Compact WY writes the product of nb reflectors as $I - VTV^{\dagger}$, so the update becomes matrix products.

MPS. NVIDIA Multi-Process Service: lets kernels of several processes run concurrently on one GPU. Without it processes are time-sliced.

CCS. Intel compute command streamer mode: splits a Max 1550 tile into several hardware queues (`ZEX_NUMBER_OF_CCS`).

Tile, card. A Max 1550 card has two tiles, each a separate device under `ZE_FLAT_DEVICE_HIERARCHY=FLAT`.

JIT, AOT. Just-in-time compilation of device code at program start, or ahead-of-time compilation for a specific device at build time.

Level Zero. Intel's low-level GPU driver interface; the staging buffer for small copies is one of its settings.

libgomp, GCN plugin. GCC's OpenMP runtime and its back end for AMD GPUs (GCN is the AMD GPU architecture family).

USM. Unified shared memory in SYCL: host memory that the device can access directly.

B File-by-file guide to the GPU source

Line counts as of 2026-11-13.

`Prog/gpu_state_mod.F90` **(1902 lines).** Setup and the device mirror. `gpu_env_read` and `gpu_env_check` read every `ALF_GPU_*` and `ALF_LA_TEST_*` variable from `/proc/self/environ`, print the build and the debug environment, and refuse debug variables in production runs. `gpu_supported` decides whether the GPU path is used. `gpu_state_setup` builds the flat mirror of hopping, checkerboard groups, vertices and fields, allocates the work arrays and maps everything to the device; it also binds the rank to a device. Contains the mirror self-test and `report_layout`, which prints all array sizes. Note: `/proc` is read before `gpu_supported`, so a GPU build stops on systems without `/proc` even with `ALF_GPU_DISABLE=1`.

`Prog/gpu_wrap_mod.F90` **(2660 lines).** Propagation: the five hopping routines with the dense `Hd` path, the checkerboard group kernels and the dense-element paths; time-dependent hopping; vertex multiplication and wraps, single and fused per slice; `Symm`; the composites used by the stabilisation. Self-tests and timing for each routine. Initialises `alf_la`.

`Prog/gpu_update_mod.F90` **(633 lines).** `gpu_wrapgrup` and `gpu_wrapgrdo`, the device versions of `WRAPGRUP`/`WRAPGRDO`: the loop over vertices, the ratio and the Metropolis decision on the host, the rank-one update kernels, and the dispatch to the low-rank module. Manages the host/device validity flags of G (`g_in`, `g_out`, `gpu_g_host`). Trace and update self-test.

`Prog/gpu_lowrank_mod.F90` **(1723 lines).** Delayed and submatrix updates on the device: the low-rank representations, proposals, accepts, wraps and flushes; the fast variants; block mode (`blk_begin`, `blk_end`, `blk_flush`, host block wraps and proposals); the asynchronous Intel block path (`ab_*`, via `libalfsb.so`); the choice of N_{d}.

`Prog/gpu_stab_mod.F90` **(2118 lines).** Stabilisation: the device stack and work slots, `gpu_stab_wrapur/ul`, decompose (default and `STABLOG`), `gpu_stab_exchange`, `CGR` with precision check and phase, the LU-free phase tracking, Newton–Schulz, the scale test, and the stabilisation self-test.

`Prog/gpu_tau_mod.F90` **(2627 lines).** The time-displaced pass on the device: propagation of $G(\tau,0)$, $G(0,\tau)$, $G(\tau,\tau)$, `CGR2_2` on $2N\times 2N$ matrices, the four precision checks, and the interface to the device τ observables. Contains extensive diagnostics (`TAUDIAG*`, a quad-precision reference comparison) used during development.

`Prog/gpu_obs_mod.F90` **(785 lines).** Equal-time and time-displaced observables of the Hubbard model on the device: scalars, correlation functions with `acc0`, point-defect local observables, `Symm`, and the per-bin flush of the accumulators.

`Libraries/Modules/alf_la_mod.F90` **(2115 lines).** The library layer: the oneMKL module `alf_la_om_mod` and the public `alf_la_mod` with host and CUDA code, `la_qrpp`, `la_geqp3_host`, info slots, stream setup and test hooks (Table 2).

`alf_la_sycl_shim.cpp` **(304 lines).** C++ shim for oneMKL LAPACK on the interop SYCL queue (`alfsh_*`): `geqrf`, `ungqr`, `unmqr`, `getrf`, `getrs`; loads `libalfdevqr.so` at run time; timing and launch benchmarks; test hooks.

`alf_la_sycl_blk.cpp` **(102 lines).** `libalfsb.so`: the asynchronous block operations (gather, scale, flush gather, GEMM, copies) on the in-order interop queue.

`alf_la_sycl_devqr.hpp` **(349),** `alf_la_sycl_qr2.hpp` **(456),** `alf_la_sycl_devqr.cpp` **(95).** `libalfdevqr.so`: the own SYCL QR, level 1 (panel with T, `ZLARFB` by GEMMs) and level 2 (T-free panel, apply kernel, device `unmqr`).

`alf_la_hip_mod.F90` **(830),** `alf_la_hip_shim.cpp` **(132).** The HIP backend of `alf_la`: hipBLAS/hipSOLVER through a C++ shim, own QR selection, `la_qrpp` with HIP kernels.

`alf_hipk_mod.F90` **(578),** `alf_hip_kernels.hip.cpp` **(621).** HIP kernels for the small operations of the update, observables and stabilisation, and the deferred-synchronisation hook for the patched GCC plugin.

`alf_hip_qr.hip.cpp` **(347).** Own HIP QR and `unmqr` (panel in registers, wave-per-columns apply).

Hooks in CPU files. `main.F90` (setup, stack initialisation, the stabilisation points of the sweep, measurement transfers, τ pass, flush per bin, finalise); `Wrapgr_mod.F90:141-149, 353-360` (early branches of `WRAPGRUP`/`DO`); `tau_m_mod.F90:52, 80` (τ pass); `Hamiltonian_Hubbard_`

smod.F90:134, 786, 831 (observables); Hop_mod.F90:301-392, Operator_mod.F90:1180-1199, control_mod.F90:457, 542, Hamiltonian_main_mod.F90:141 (read-only accessors and the precision routines on scalars). Lowrank_update_mod.F90 and upgrade_mod.F90 have no GPU code.

Build files. configure.sh (GPU= switch, machine profiles PGI, HELMAGPU, IntelLLVM, GNU); Prog/Makefile:16-24 (GPU modules, a stamp file so that the tree can switch backend without make clean); Libraries/Modules/Makefile:39-70 (SYCL and HIP objects and libraries).

C Build lines

Configure. From docs/gpu/README.md:

```
source configure.sh PGI MPI GPU=nvidia HDF5 # NVIDIA (HELMAGPU on Helma)
source configure.sh IntelLLVM MPI GPU=intel HDF5 # Intel
source configure.sh GNU MPI GPU=amd HDF5 # AMD
make lib program
```

Parameter-parallel runs replace MPI by PP. A build without GPU= or with GPU=none is the unchanged CPU code.

Flags added by configure.sh (configure.sh:611-647).

- nvidia: -mp becomes -mp=gpu -gpu=${ALF_GPU_CC:-cc80} -DALF_GPU -DALF_LA_CUDA; link -mp=gpu -gpu=... -cudalib=cublas,cusolver; base flags -Mpreprocess -O3 -Mfprelaxed -fast. HELMAGPU loads nvhpc/25.7 and openmpi/5.0.8-nvhpc25.7-cuda and sets ALF_GPU_CC=cc90,cuda12.9.
- intel: -fiopenmp -fopenmp-targets=spir64 -I${MKLROOT}/include -DALF_GPU -DALF_LA_ONEMKL, without -static; link -qmkl -lmkl_sycl_blas -lmkl_sycl_lapack -lsycl -lstdc++ -ldl -lalfsb with an rpath to Libraries/Modules. The SYCL shim is compiled with icpx -fsycl; libalfsb.so and libalfdevqr.so as shared libraries; ahead-of-time compilation of these with ALF_SYCL_BLK_FLAGS="-fsycl-targets=spir64_gen -Xs '-device pvc'".
- amd: -foffload=amdgcn-amdhsa -foffload-options=amdgcn-amdhsa=-march=${ALF_AMD_ARCH:-gfx90a} -DALF_GPU -DALF_LA_HIP, without -static and -ffast-math; link -lhipsolver -lhipblas -lamdhip64 -lstdc++; HIP kernels with hipcc -O2 -offload-arch=gfx90a, the shim with g++ -D__HIP_PLATFORM_AMD__.

Platform recipes.

```
# Helma (H100), helma_h100.md section 2
export ALF_HDF5_DIR=.../hdf5_257
. ./configure.sh HELMAGPU PP GPU=nvidia HDF5 NO-INTERACTIVE; make lib program
# SuperMUC-NG Phase 2 (Max 1550), intel_pvc_supermuc_ng2.md; oneAPI 2025.1 (intel-
    toolkit/2025.1.0)
AOT=1 bash -l build.sh prodaot IntelLLVM MPI GPU=intel HDF5 NO-INTERACTIVE
# AMD MI210: patched libgomp GCN plugin first in LD_LIBRARY_PATH, GCN_NUM_THREADS=4
```

Run.

```
OMP_TARGET_OFFLOAD=MANDATORY OMP_NUM_THREADS=1 mpirun -n N --bind-to core ./ALF.out
# Intel additionally:
ZE_FLAT_DEVICE_HIERARCHY=FLAT LIBOMPTARGET_LEVEL_ZERO_STAGING_BUFFER_SIZE=1048576
    ALFSH_ASYNCBLK=1
```

The info file must contain "GPU path (gpu_state): ACTIVE".

Code and documents disagree. BUILD.md lists only GPU=none|nvidia|intel, gives nvhpc 24.11 and CUDA 12.6 for Helma (the code defaults to 25.7 and 12.9), omits the SYCL libraries from the Intel link line, and calls ahead-of-time compilation a future ticket.

D Run-time switches

The complete list with source lines is `docs/gpu/RUNTIME_SWITCHES.md`. Table 18 gives the ones that matter for production and the main debug switches, checked against the code.

Table 18: Run-time switches (environment variables). "Guarded": refused in a production run unless `ALF_GPU_DEBUG` is set.

Name	Values	Effect
`OMP_TARGET_OFFLOAD`	`MANDATORY`	stop instead of silently running on the CPU if no device
`ALF_GPU_DISABLE`	`1`	run the CPU path in a GPU build
`ALF_GPU_DEBUG`	not `0`	declares a debug run; lifts the guard
`ALF_UNSTABLE_THRESHOLD`	real, default 10	abort threshold of the precision check of G (production: 10^{-4})
`ALF_GPU_OBS_SKIP`	list of 1..4	equal-time observables not measured (stay zero)
`ALF_GPU_NOOBS`, `ALF_GPU_NOUPDATE`	`1`	host observables / host update (A/B tests); guarded
`ALF_GPU_SELFTEST`	1, 2, 3, 4	mirror, wrap, stabilisation, τ self-tests; guarded
`ALF_GPU_UPDTEST`, `_TIMING`, `_STABTIME`, `_TAUTIME`, `_EXIT`	`1`	update test, timing hooks, exit after a hook; guarded
`ALF_GPU_NS`, `ALF_GPU_TAU_NS`	`0`	switch off the Newton–Schulz step; guarded
`ALF_GPU_CGR_LU`	1, 2	always LU phase / both and compare; guarded
`ALF_GPU_FAULT`, `_FAULT_STAB`, `_FAULT_TAU`, `_FAULT_TAU_AT`	see text	planted defects; guarded
`ALF_GPU_STABCHECK`, `_STABMIX`, `_TAUSWAP`, `_TAUDIAG*`, `_TRACE`		host checks, isolation and diagnostic experiments; guarded
`ALF_GPU_TL`, `_TR`, `_VB`	integers 1, 2, 8	columns or rows per thread of the wrap kernels; guarded
`LIBOMPTARGET_LEVEL_ZERO_STAGING_BUFFER_SIZE`	`1048576`	Intel: 32 KB copies 380→22 μs
`ZE_FLAT_DEVICE_HIERARCHY`	`FLAT`	Intel: each tile is a device
`ZEX_NUMBER_OF_CCS`	`0:4,...,7:4`	Intel: four hardware queues per tile
`ALFSH_ASYNCBLK`	`1`	Intel: asynchronous block path
`ALFSH_DEVQR`	1, 2	Intel: own device QR (level 1, level 2)
`ALFSH_DEVUNMQR`	`1`	Intel: own device `unmqr`
`ALFSH_QRNB`, `ALFDEVQR_*`	integers	Intel: QR block sizes, kernel variants
`ALFSH_TIMING`, `ALFSH_LAUNCHBENCH`		Intel: per-routine timing, launch benchmark
`ALF_HIP_QR`	`0`	AMD: hipSOLVER instead of the own QR
`ALF_HIP_ASYNC`	`0`	AMD: synchronise after every HIP call
`GCN_NUM_THREADS`	4	AMD: threads of the libgomp GCN runtime
`CUDA_VISIBLE_DEVICES`, `CUDA_MPS_*`		NVIDIA: rank binding and MPS share

The guard covers only variables named `ALF_GPU_*` and `ALF_LA_TEST_*`. The `ALFSH_*`, `ALF_HIP_*` and `ALFDEVQR_*` variables are read by C++ code and are not guarded; `GPU_PLAN_V2.md` (A9) proposes to print each of them into `info`.

Code and documents disagree. `RUNTIME_SWITCHES.md` (line 82) says `ALF_GPU_SELFTEST` runs the mirror self-test for any value except 0, 2, 3, 4; the code excludes only 3 and 4, so `SELFTEST=2` runs the mirror self-test and then the wrap self-test. `ALF_GPU_CGR_LU` is not in the document. `ALF_GPU_DEBUG` declares a debug run for any value other than `0`, not only `1`.